



Biblioteca de algoritmos e códigos em C++ para consulta nas maratonas de programação

Esta biblioteca foi criada para consulta dos principais algoritmos utilizados nas maratonas de programação, esses algoritmos foram implementados utilizando a linguagem C++, visando:

1. Auxiliar na aprendizagem de programação e suas aplicações;
2. Desenvolver soluções eficientes para resolver problemas computacionais;
3. Compreender algoritmos e suas técnicas;
4. Incentivar a participação em maratonas;
5. O trabalho tem como objetivo:
 - (a) *Motivar os alunos em competições*
 - (b) *Criar um material impresso que possa ser utilizado:*

1. Complexidade

O que é a complexidade de um algoritmo?

Complexidade de algoritmo refere-se à medida de eficiência de um algoritmo em termos de tempo e espaço. Ela avalia o quanto de recurso computacional (como CPU ou memória) que um algoritmo requer para resolver o problema conforme sua entrada cresce. A complexidade é geralmente expressa na notação Big O, que descreve o comportamento assintótico do algoritmo ao considerar o tamanho da entrada para um pior caso. Por exemplo, um algoritmo com complexidade $O(n)$ tem um tempo de execução que cresce linearmente com o tamanho da entrada, enquanto um com complexidade $O(n^2)$ tem um tempo de execução com crescimento quadrático.

Existem dois tipos principais de complexidade: complexidade de tempo e complexidade de espaço. A complexidade de tempo se refere à quantidade de tempo que um algoritmo leva para processar uma entrada de tamanho n , enquanto a complexidade de espaço se refere à quantidade de memória necessária. Entender a complexidade ajuda na escolha de algoritmos mais eficientes, especialmente para grandes conjuntos de dados, permitindo soluções que equilibram entre rapidez e uso de recursos, fundamental para otimização em desenvolvimento de software e ciência da computação.

Notação O

Quando existe um limite assintótico superior, a notação O é utilizada. Para uma dada função $g(n)$, denotamos por $O(g(n))$ (lê-se "Ó grande de g de n " ou, "ó de g de n ") o conjunto de funções $O(g(n)) = f(n)$ (existem constantes positivas c e n_0 tais que $0 \leq f(n) \leq c * g(n)$ para todo $n \geq n_0$).

Usamos a notação O para dar um limite superior a uma função, dentro de um fator constante. Para todos os valores n em n_0 ou à direita de n_0 , o valor da função $f(n)$ está abaixo de $c * g(n)$.

Tendo em vista que a notação O descreve um limite superior, quando a empregamos para limitar o tempo de execução do pior caso de um algoritmo temos um limite para o tempo de execução do algoritmo em cada entrada $[o]$.

1.1. Complexidade $O(1)$

A complexidade $O(1)$, ou complexidade de tempo constante, significa que um algoritmo leva o mesmo tempo para executar, independentemente do tamanho da entrada. Isso ocorre porque o algoritmo realiza um número fixo de operações, como calcular a distância entre dois pontos ou acessar um elemento específico em matrizes.

A fim de demonstrar o sistema de complexidade $O(1)$ será utilizado um exemplo como base, neste exercício é necessário criar um programa que quantifique diferentes tipos de pisos para uma sala qualquer, onde não basta apenas ter área da sala, pois os pisos se diferem entre inteiros e cortados, então leva-se em consideração os espaços não completos do chão da sala.

A complexidade $O(1)$, trabalha com uma constante durante todo o seu processo, para fins de contagem a constante é desprezada. Segue o exemplo:

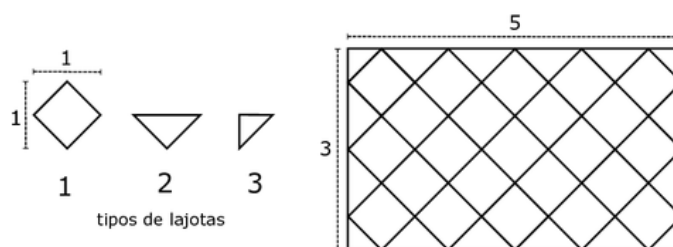
Piso da escola

OBI 2018

Timelimit: 1

O colégio pretende trocar o piso de uma sala de aula e a diretora aproveitou a oportunidade para passar uma tarefa aos alunos. A sala tem o formato de um retângulo de largura L metros e comprimento C metros, onde L e C são números inteiros. A diretora precisa comprar lajotas de cerâmica para cobrir todo o piso da sala. Seria fácil calcular quantas lajotas seriam necessárias se cada lajota fosse um quadrado de 1 metro de lado. O problema é que a lajota que a diretora quer comprar é um quadrado que possui 1 metro de diagonal, não de lado. Além disso, ela quer preencher o piso da sala com as diagonais das lajotas alinhadas aos lados da sala, como na figura.

A loja vai fornecer lajotas do tipo 1: inteiras; do tipo 2, que correspondem à metade das do tipo 1, cortadas ao longo da diagonal; e lajotas do tipo 3, que correspondem à metade do tipo 2. Veja os três tipos de lajotas na figura.



Está muito claro que sempre serão necessárias 4 lajotas do tipo 3 para os cantos da sala. A tarefa que a diretora passou para os alunos é calcular o número de lajotas dos tipos 1 e 2 que serão necessárias. Na figura, para $L = 3$ e $C = 5$, foram necessárias 23 do tipo 1 e 12 do tipo 2.

Seu programa precisa computar, dados os valores de L e C , a quantidade de lajotas do tipo 1 e do tipo 2 necessárias.

Entrada

A entrada consiste de um único número inteiro L indicando a largura da sala. A segunda linha contém um inteiro C representando o comprimento da sala, conforme dados da tabela 1.

Saída

Imprima duas linhas na saída. A primeira deve conter um inteiro, representando o número de lajotas do tipo 1 necessárias. A segunda deve conter um inteiro, indicando o número de lajotas do tipo 2.

Restrições

- $1 \leq L, C \leq 100$

$$T(n) = (C_1 * 1) + (C_2 * 1) + (C_3 * 1) + (C_4 * 1) + (C_5 * 1) + (C_6 * 1) + (C_7 * 1) + (C_8 * 1) \quad (1)$$

$$T(n) = (C_1 + C_2 + C_3 + C_4 + C_5 + C_6 + C_7 + C_8) * 1 \quad (2)$$

Exemplos de Entrada	Exemplos de Saída
3	23
5	12
1	1
1	0

Table 1. Exemplos de entradas e saídas

Algoritmo	Custo	Complexidade
Algoritmo 1: Piso da Sala	C1	O(1)
1 início	C2	O(1)
2 Declare L, C, T1, T2 inteiro	C3	O(1)
3 Leia T1, T2	C4	O(1)
4 $T1 \leftarrow L * C + (L - 1) * (C - 1)$	C5	O(1)
5 $T2 \leftarrow (L - 1) * 2 + (C - 1) * 2$	C6	O(1)
6 Escreva T1	C7	O(1)
7 Escreva T2	C8	O(1)
8 fim	K	O(1)

$$Considere\ K = C_1 + C_2 + C_3 + C_4 + C_5 + C_6 + C_7 + C_8$$

(3)

$$T(n) = K * 1 = O(1)$$

(4)

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int L, C, T1, T2;
6
7      cin >> L >> C;
8
9      T1 = L*C + (L-1)*(C-1);
10     T2 = (L-1)*2 + (C-1)*2;
11
12     cout << T1 << endl;
13     cout << T2 << endl;
14
15     return 0;
16 }
```

1.2. Complexidade $O(n)$

Uma complexidade $O(n)$ (complexidade linear) significa que o tempo ou espaço necessário para executar um algoritmo aumenta de forma linear com o tamanho da entrada. Em outras palavras, se você dobrar o tamanho da entrada, o algoritmo também dobrará em tempo de execução ou espaço de memória.

Para demonstrar a complexidade $O(n)$ um exercício retirado do site e [e] de busca em um vetor foi escolhido, a atividade em questão consiste em procurar dentro de uma lista de números qual é o menor valor entre eles, destacando também a sua posição dentro da lista.

Menor e Posição

BeeCrowd 1180

Timelimit: 1

Faça um programa que leia um valor N . Este N será o tamanho de um vetor $X[N]$. A seguir, leia cada um dos valores de X , encontre o menor elemento deste vetor e a sua posição dentro do vetor, mostrando esta informação.

Entrada

A primeira linha de entrada contém um único inteiro N ($1 < N < 1000$), indicando o número de elementos que deverão ser lidos em seguida para o vetor $X[N]$ de inteiros. A segunda linha contém cada um dos N valores, separados por um espaço. Vale lembrar que nenhuma entrada haverá números repetidos.

Saída A primeira linha apresenta a mensagem “Menor valor:” seguida de um espaço e do menor valor lido na entrada. A segunda linha apresenta a mensagem “Posição:” seguido de um espaço e da posição do vetor na qual se encontra o menor valor lido, lembrando que o vetor inicia na posição zero.

Exemplos de Entrada	Exemplos de Saída
10	Menor valor: -5
1, 2, 3, 4, -5, 6, 7, 8, 9, 10	Posicao: 4

Table 2. Exemplos de entradas e saídas

$$T(n) = C_1 * 1 + C_2 * 1 + C_3 * 1 + C_4 * (n + 1) + C_5 * n + C_6 * 1 + C_7 * 1 + C_8 * n \quad (5)$$

$$+ C_9 * (n - 1) + C_{10} * (n - 1) + C_{11} * 1 + C_{12} * 1 + C_{13} * 1 + C_{14} * 1 + C_{15} * 1 \quad (6)$$

$$T(n) = K * (1 + 1 + 1 + n + 1 + n + n + 1 + n + n - 1 + n - 1 + n - 1 + n - 1 + 1 + 1 + 1) \quad (7)$$

$$T(n) = 8 * n + 4 = O(n) \quad (8)$$

Table 3. Exemplos de entradas e saídas

Algoritmo	Custo	Complexidade
Algoritmo 2: Menor e Posicao		
1 Declare X, N, M, POS inteiro;	C1	1
2 Leia N ;	C2	1
3 for $X \leftarrow 0$ <i>até</i> $N - 1$ <i>faça do</i>	C3	1
4 Leia $P[X]$;	C4	1
5 end	C5	n
6 $M \leftarrow 0$;	C6	n
7 for $X \leftarrow 1$ <i>até</i> $N - 1$ <i>faça do</i>	C7	1
8 if $P[X] < P[M]$ then	C8	$n - 1$
9 $M \leftarrow X$;	C9	$n - 1$
10 end	C10	$n - 1$
11 end	C11	$n - 1$
12 Escreva $P[M]$;	C12	1
13 Escreva M ;	C13	1

```

1  #include <bits/stdc++.h>
2
3
4  int main() {
5
6      int N;
7      std::cin >> N;
8      int X[1000];
9      int menor;
10     int posicao;
11
12     for(int i = 0; i < N; i++) {
13
14         std::cin >> X[i];
15     }
16
17     menor = X[0];
18     posicao = 0;
19
20     for(int i = 1; i < N; i++) {
21
22         if(X[i] < menor) {
23             menor = X[i];
24             posicao = i;
25         }
26     }
27     std::cout << "Menor valor: " << menor
28               << std::endl;
29     std::cout << "Posicao: " << posicao <<
30               << std::endl;
31     return 0;
32 }

```

1.3. Complexidade $O(n^2)$

A complexidade $O(n^2)$ é uma medida de desempenho de algoritmos que indica que o tempo de execução cresce quadraticamente com o tamanho da entrada, representado por ' n '. Isso significa que, se o tamanho da entrada dobrar, o tempo de execução será aproximadamente quatro vezes maior, devido ao fator quadrático. Algoritmos com essa complexidade frequentemente envolvem dois loops aninhados, onde cada elemento da entrada é comparado ou processado em relação a todos os outros elementos.

Um exemplo clássico de algoritmo com complexidade $O(n^2)$ é o Bubble Sort, onde, para cada elemento de uma lista, são feitas comparações com os elementos subsequentes para realizar trocas e ordenar os dados. Em um caso médio ou pior, o algoritmo precisa percorrer a lista n vezes, e em cada iteração, realiza até n comparações ou operações, resultando em $n * n = n^2$ operações. Essa característica torna esses algoritmos ineficientes para grandes volumes de dados, pois o custo computacional aumenta rapidamente.

Embora algoritmos $O(n^2)$ sejam menos eficientes do que aqueles com complexidades lineares $O(n)$ ou logarítmicas $O(\log n)$, eles ainda têm utilidade em cenários com entradas pequenas ou quando a simplicidade de implementação é mais importante que o desempenho. Além disso, podem servir como base para otimizações ou em situações específicas onde o pior caso raramente ocorre. No entanto, para aplicações de larga escala, é comum buscar alternativas mais eficientes, como algoritmos com complexidade $O(n \log n)$, típicos de métodos de ordenação mais avançados como Quick Sort ou Merge Sort.

Segue o exemplo representativo da complexidade $O(n^2)$:

Fibonacci em Vetor

Beecrowd 1176

Timelimit: 1

Faça um programa que leia um valor e apresente o número de Fibonacci correspondente a este valor lido. Lembre que os 2 primeiros elementos da série de Fibonacci são 0 e 1 e cada próximo termo é a soma dos 2 anteriores a ele. Todos os valores de Fibonacci calculados neste problema devem caber em um inteiro de 64 bits sem sinal.

Entrada

A primeira linha da entrada contém um inteiro T , indicando o número de casos de teste. Cada caso de teste contém um único inteiro N ($0 \leq N \leq 60$), correspondente ao N -ésimo termo da série de Fibonacci.

Saída

Para cada caso de teste da entrada, imprima a mensagem " $\text{Fib}(N) = X$ ", onde X é o N -ésimo termo da série de Fibonacci.

Exemplos de Entrada	Exemplos de Saída
3 0 4 2	FIB(0) = 0 FIB(4) = 3 FIB(2) = 1

Table 4. Exemplos de entradas e saídas

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int T;
6      cin >> T;
7      unsigned long long fib[61];
8
9      fib[0] = 0;
10
11     fib[1] = 1;
12
13     for(int i = 2; i <= 60; i++) {
14         fib[i] = fib[i-1] + fib[i-2];
15     }
16
17     for(int i = 0; i < T; i++) {
18         int N;
19         cin >> N;
20         cout << "Fib(" << N << ") = " <<
                fib[N] << endl;
21     }
22
23     return 0;
24 }

```

$$T(n) = C1 * 1 + C2 * 1 + C3 * 1 + C4 * (59) + C5 * T + C6 * T$$

$$T(n) = K + (C5 + C6) * T$$

$$T(n) = a * T + b = O(T)$$

2. Abóbora Contest

Como meio didático e objeto de estudos sobre as maratonas de programação, serão utilizadas as questões da maratona do Abóbora Contest (Maratona própria do Instituto Federal Goiano Campus Rio Verde). Nesta seção serão apresentadas as questões presentes nas provas dos anos de 2022, 2023, 2024 e 2025. Incluindo, também, suas respectivas soluções.

Informações Gerais

Cada caderno contém 10 problemas exclusivos criados por professores e alunos do Instituto Federal Goiano - Campus Rio Verde.

Segue abaixo as regras da respectiva maratona:

1. Sobre os nomes dos programas

- (a) Para soluções em C/C++ e Python 3.8, o nome do arquivo-fonte não é significativo, pode ser qualquer nome desde que tenha as extensões .c, .cc, .py3.
- (b) Para linguagem Java sua solução deve ser chamado de `codigo_do_problema.java`, onde `codigo_do_problema` é a letra maiúscula que identifica o problema. Lembre que em Java o nome da classe principal deve ser igual ao nome do arquivo.

2. Sobre a entrada

- (a) A entrada de seu programa deve ser lida da entrada padrão.
- (b) A entrada é composta de um único caso de teste, descrito em um número de linhas que depende do problema.
- (c) Quando uma linha da entrada contém vários valores, estes são separados por um único espaço em branco; a entrada não contém nenhum outro espaço em branco.
- (d) Cada linha, incluindo a última, contém exatamente um caractere final-de-linha.
- (e) O final da entrada coincide com o final do arquivo.

3. Sobre a saída

- (a) A saída de seu programa deve ser escrita na saída padrão.
- (b) Quando uma linha da saída contém vários valores, estes devem ser separados por um único espaço em branco; a saída não deve conter nenhum outro espaço em branco.
- (c) Cada linha, incluindo a última, deve conter exatamente um caractere final-de-linha.

Maratona de Programação Abóbora Contest 2022

Problema A

Marlus e a Fila

by André

Timelimit: 1

Marlus é um coordenador de curso muito dedicado do curso de computação do NuCAT. Os alunos estão enfrentando um grande problema na entrada do NuCAT, todos querem entrar no mesmo horário gerando assim um grande tumulto. Marlus reuniu com os cavalheiro do NDE para tentar resolver o problema. Depois de muita discussão o NDE resolveu ordenar os alunos pelo número de matrícula, e organizando a entrada pela ordem crescente. Mas Marlus precisa de sua ajuda para desenvolver o programa pois ele está muito ocupado com seu novo doutorado.

Entrada Para cada dia será lido um inteiro N , $1 \leq N \leq 10^4$, que representa o número de alunos disponíveis na entrada. Depois será lido os N números de matrículas.

Saída Para cada dia, imprimir os número de matrículas ordenados em ordem crescente.

Exemplos de Entrada	Exemplos de Saída
5 5 4 3 2 1	1 2 3 4 5

Table 5. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int ler() {
5      int num;
6      cin >> num;
7      return num;
8  }
9
10 void vetor(int n, vector<int> &vet) {
11     for (int i = 0; i < n; i++) {
12         vet[i] = ler();
13     }
14 }
15
16 void escrever(int n, const vector<int> &vet) {
17     for (int i = 0; i < n; i++) {
18         cout << vet[i] << " ";
19     }
20     cout << "\n";
21 }
22
23 void troca(int &x, int &y) {
24     int aux = x;
25     x = y;
26     y = aux;
27 }
28
29 void ordena(int n, vector<int> &vet) {
30     for (int i = 0; i < n - 1; i++) {
31         int menor = i;
32         for (int j = i; j < n; j++) {
33             if (vet[j] < vet[menor]) {
34                 menor = j;
35             }
36         }
37         troca(vet[menor], vet[i]);
38     }
39 }
40
41 int main() {
42     ios::sync_with_stdio(false);
43     cin.tie(nullptr);
44
45     int n = ler();
46     vector<int> vet(n);
47
48     vetor(n, vet);
49     ordena(n, vet);
50     escrever(n, vet);
51
52     return 0;
53 }
```

Problema B

Vida Artificial

Por Heverton Barros de Macêdo

Timelimit: 1

Charles é um grande pesquisador sobre os seres vivos e agora passou a investigar modelos de vida artificial.

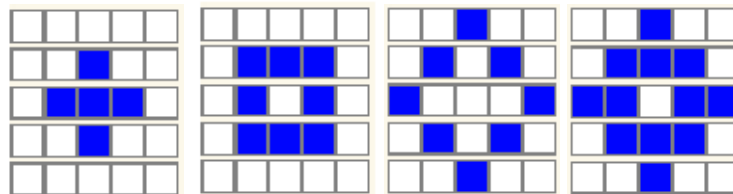
O principal modelo estudado por Charles é composto por uma matriz bidimensional de tamanho $M \times N$, em que cada célula está em um de dois possíveis estados (morta ou viva). A vizinhança de uma célula é formada por 8 vizinhos adjacentes (considerando as células da diagonal).

As regras do modelo são simples e elegantes:

1. Qualquer célula viva com menos de dois vizinhos vivos morre de solidão.
2. Qualquer célula viva com mais de três vizinhos vivos morre de superpopulação.
3. Qualquer célula morta com exatamente três vizinhos vivos se torna uma célula viva.
4. Qualquer célula viva com dois ou três vizinhos vivos continua no mesmo estado para a próxima geração.

Nesse modelo todos os nascimentos e mortes ocorrem simultaneamente e representam uma geração na história da vida da configuração inicial.

Para ilustrar graficamente como o modelo de vida artificial funciona, considere a seguinte sequência de figuras.



A primeira figura (da esquerda para direita) representa uma configuração inicial para o modelo, as figuras seguintes foram obtidas como resultado dos passos de evolução até alcançar a terceira geração.

Ajude Charles a simular o modelo de vida artificial citado acima.

Entrada

A entrada contém vários casos de teste. A primeira linha de cada caso corresponde a quantidade de gerações a ser evoluída, denominada aqui como Q ($1 \leq Q \leq 100$). A segunda linha indica a dimensão de linha ($3 \leq M \leq 100$) e coluna ($3 \leq N \leq 100$) da matriz bidimensional, separados por um espaço. A terceira linha possui os estados iniciais da matriz representados de forma linear (primeiro elemento, segundo elemento e assim sucessivamente), sendo o caractere 0 o estado para células mortas e 1 para células vivas.

O final da entrada é indicado por $Q = 0$.

Saída

O programa deve produzir, de forma linear, os estados de cada célula da matriz após passadas Q gerações.


```
42 void evol(vector<vector<char>> &m, int qtd_lin, int qtd_col, int dim_i, int dim_j){
43     vector<vector<char>> aux(qtd_lin, vector<char>(qtd_col));
44     for(int i = 0; i < qtd_lin; i++){
45         for(int j = 0; j < qtd_col; j++){
46             aux[i][j] = aplicar_regra(m, i, j, dim_i, dim_j);
47         }
48     }
49     m = aux;
50 }
51
52 int main(){
53     ios::sync_with_stdio(false);
54     cin.tie(nullptr);
55
56     int qtd_ger, qtd_lin, qtd_col;
57
58     while(true){
59         if(!(cin >> qtd_ger)) break;
60         if(qtd_ger <= 0) break;
61
62         cin >> qtd_lin >> qtd_col;
63
64         vector<vector<char>> m(qtd_lin, vector<char>(qtd_col));
65
66         for(int i = 0; i < qtd_lin; i++){
67             for(int j = 0; j < qtd_col; j++){
68                 cin >> m[i][j];
69             }
70         }
71
72         for(int i = 0; i < qtd_ger; i++){
73             evol(m, qtd_lin, qtd_col, qtd_lin, qtd_col);
74         }
75
76         saida_mat(m, qtd_lin, qtd_col);
77         cout << "\n";
78     }
79
80     return 0;
81 }
```

Problema C

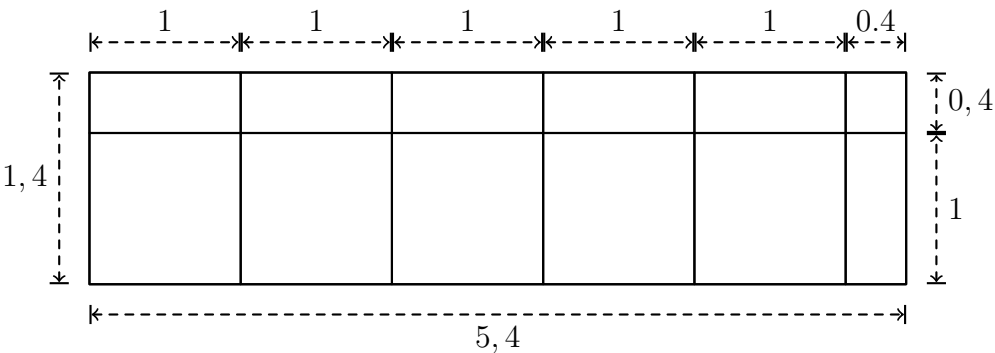
Revestimento da Piscina

Por Márcio Belo-Filho

Timelimit: 1

O professor Márcio pretende fazer uma piscina na sua casa. A piscina é retangular e suas paredes e fundo serão revestidos com azulejos quadrados. Ao final do revestimento alguns azulejos estarão completos e outros estarão cortados, sendo sempre utilizado uma das partes cortadas e outra parte descartada. O professor Márcio quer saber quantos azulejos serão utilizados na piscina e a área total descartada (medida em azulejos).

A figura abaixo apresenta um exemplo de uma dos lados da piscina, de 5,4 metros de comprimento por 1,4 metros de altura, com azulejos de 1 metro de lado.



Note que foram utilizados 12 azulejos nessa lateral da piscina, sendo 5 azulejos inteiros e 7 cortados. Além disso, foram descartados 4.44 azulejos.

Entrada A entrada contém um único caso de teste. A única linha de cada caso de teste contém quatro números reais (de no máximo três casas decimais): (1) o lado do azulejo quadrado $0.05 \leq Q \leq 10$; (2) o comprimento da piscina $1 \leq C \leq 50$; (3) a largura da piscina $1 \leq L \leq 25$; e (4) a profundidade da piscina $0.5 \leq P \leq 5$, nessa ordem. Todas as medidas estão em metros.

Saída Para cada caso de teste da entrada seu programa deve imprimir uma única linha na saída, contendo o número de azulejos usado na piscina e a área total descartada (medida em azulejos).

Exemplos de Entrada	Exemplos de Saída
1 1 1 1	5 0.0
1 5.4 5.4 1.4	84 24.60
1.2 5 4 1	38 11.61

Table 7. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      double q, c, l, p;
9      if (!(cin >> q >> c >> l >> p)) return 0;
10
11     double tc = ceil(c / q);
12     double tl = ceil(l / q);
13     double tp = ceil(p / q);
14
15     double T = tc * tl + 2 * tc * tp + 2 * tl * tp;
16     double D = T - (c * l + 2 * c * p + 2 * l * p) / (q * q);
17
18     cout << fixed << setprecision(0) << T << " ";
19     cout << fixed << setprecision(2) << D << "\n";
20
21     return 0;
22 }
```


Problema D

O Teorema de Lis

Por Athos José

Timelimit: 1

Se o assunto é relacionamento, o tipo de garota de Colin Singleton tem nome: Lis. E, em se tratando de Colin e Lis, o desfecho é sempre o mesmo, ele leva um belo fora. Já aconteceu e foram muitos.

Após o mais recente e traumático pé na bunda, o Colin que só namora Lis resolve cair de cara na programação. Com seu caderninho de anotações no bolso e o melhor amigo (você) do lado que o está ajudando na programação, o garoto PhD em levar o fora, descobre sua verdadeira missão. Se você acha que é elaborar e comprovar o Teorema Fundamental da Previsibilidade das Lis? A resposta é não! Mas sim encontrar o tamanho de uma subsequência de idades das Lis que Colin namorou.

Dadas as idades que estão no caderninho de anotações de Colin, você deve encontrar o tamanho da subsequência de idades crescente que seja a mais longa possível. Esta subsequência não é necessariamente contígua.

Entrada

A entrada é composta por vários casos de teste. A Primeira linha é a quantidade de casos de teste T ($1 \leq T \leq 400$).

Os próximos T casos de teste é composto por um número N onde ($1 \leq N \leq 400$), indicando a quantidade de idades anotadas. A próxima linha contem N números que compõem um array de *idades* onde ($18 \leq idades_i \leq 50$).

Saída

A saída deve conter apenas um número que é a quantidade de elementos da maior subsequência.

Exemplos de Entrada	Exemplos de Saída
3 5 18 20 19 26 25 10 34 47 23 29 50 21 22 48 32 34 7 18 20 31 33 39 43 44	3 4 7
2 19 24 36 26 27 25 29 48 44 22 43 40 37 45 48 21 19 19 21 48 4 25 31 38 31	7 3

Table 8. Exemplos de entradas e saídas

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  typedef vector<int> vi;
6
7  int lis(vi &a, int n){
8      vi d(n, 1);
9      for(int i = 0; i < n; i++){
10         for(int j = 0; j < i; j++){
11             if(a[j] < a[i]){
12                 d[i] = max(d[i], d[j]+1);
13             }
14         }
15     }
16     int ans;
17     for(int i = 0; i < n; i++){
18         ans = max(ans, d[i]);
19     }
20     return ans;
21 }
22
23 int main(){
24     int n, ans, t;
25     cin >> t;
26     while(t--){
27         cin >> n;
28         vi a(n, 0);
29         for(int i = 0; i < n; i++){
30             cin >> a[i];
31         }
32         ans = lis(a, n);
33         cout << ans << endl;
34     }
35     return 0;
36 }

```

Problema E

A Mudança

Por Douglas Cedrim

Timelimit: 1

O ano é 2019, segundo semestre, e o curso de computação terá finalmente seu prédio inaugurado. Para conseguir tal feito, será preciso migrar todos os C computadores disponíveis para o novo prédio. Motivados para inaugurar o seu novo lugar de estudos, os estudantes mobilizam um total de V voluntários para ajudar no processo, carregando cada qual um certo número de computadores do prédio antigo para o novo prédio.

De forma a evitar o desgaste individual e dividir o trabalho de forma mais balanceada possível, os alunos reúnem-se sob a sombra de um ipê amarelo e, diferente dos pré-socráticos na Grécia antiga, decidem debater e deixar por escrito um algoritmo que resolve esse problema de forma otimizada para que, no dia da mudança, seja executado e a distribuição seja feita da melhor forma possível.

Como eles poderiam resolver esse problema, de forma que a distribuição do número de computadores que cada voluntário deverá carregar seja mais balanceada possível?

Entrada O arquivo de entrada possui 1 linha. Ela contém dois números inteiros V e C ($1 \leq V \leq 10^3$) e ($1 \leq C \leq 10^2$), indicando o número de voluntários e a quantidade máxima de computadores que devem ser levados de um prédio para outro, respectivamente.

Saída Seu programa deve imprimir uma única linha na saída, contendo números inteiros maiores ou iguais a zero, separados por espaço, indicando quantos computadores cada voluntário deverá levar para o novo bloco.

Exemplos de Entrada	Exemplos de Saída
8 32	4 4 4 4 4 4 4 4
13 40	4 3 3 3 3 3 3 3 3 3 3 3 3
2 80	40 40
80 2	1 1 0

Table 9. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n_nodes, n_threads;
9      cin >> n_nodes >> n_threads;
10
11     vector<int> result(n_nodes, 0);
12     int pc = 0;
13
14     while (n_threads-- > 0) {
15         result[pc]++;
16         pc = (pc + 1) % n_nodes;
17     }
18
19     for (int i = 0; i < n_nodes - 1; i++) {
20         cout << result[i] << " ";
21     }
22     cout << result[n_nodes - 1] << "\n";
23
24     return 0;
25 }
```

Problema F

Zé do Grafo

Por André da Cunha Ribeiro

Timelimit: 1

Zé do Grafo é um famoso Mestre Jedi matador de monstros. Diz a lenda que existe uma cidade escondida nos confins do IFGoiano - Campus Rio Verde. A cidade é composta por várias pontes que ligam vários bairros da cidade, muito parecido com as setes pontes da cidade de Königsberg. Mas Zé está enfrentando um grande problema na cidade que é o aumento dos monstros.

Mas Zé do Grafo tem contigo um grande poder. Esse poder permitir ele isolar todos os monstros em alguma ilha e depois ele pode destruir as pontes que ligam o bairro. Tudo estaria certo se Zé do Grafo não tivesse o limite de uma bomba. Contudo, ele precisa de sua ajuda para identificar se a cidade possui um bairro ligado por apenas uma ponte.

Ou seja, dadas as descrições dos bairros e das pontes, sua tarefa é determinar se Zé do Grafo consegue separar todos os monstros em um bairro.

Entrada A entrada contém um caso de teste. A primeira linha do caso de teste contém dois inteiros N , M , indicando respectivamente o número de bairros ($1 \leq N \leq 50$) e de pontes ($1 \leq M \leq 200$).

Cada uma das M linhas seguintes composta de dois inteiros que descreve uma ponte.

Saída Para o caso de teste da entrada seu programa deve produzir uma linha na saída contendo 'S' se Zé do Grafo consegue separar os monstros em um bairro e 'N' caso contrário.

Exemplos de Entrada	Exemplos de Saída
3 2 1 2 2 3	S
6 7 1 2 1 3 2 3 3 4 4 5 4 6 5 6	S
3 3 1 2 1 3 2 3	N

Table 10. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  void leituraGrafo(vector<vector<int>> &G, int m)
6  {
7      int a, b;
8      while (m--)
9      {
10         cin >> a >> b;
11         int u = a - 1;
12         int v = b - 1;
13         G[u].push_back(v);
14         G[v].push_back(u);
15     }
16 }
17
18 void apaga_aresta(vector<vector<int>> &G, int u, int v)
19 {
20     G[u].erase(find(G[u].begin(), G[u].end(), v));
21     G[v].erase(find(G[v].begin(), G[v].end(), u));
22 }
23
24 void MostrarArestas(vector<vector<int>> &G, int n)
25 {
26     for (int u = 0; u < n; u++)
27     {
28         cout << u << ": ";
29         for (auto v : G[u])
30         {
31             cout << v << " ";
32         }
33         cout << endl;
34     }
35 }
36
37 void dfs(vector<vector<int>> &G, int u, vector<bool> &visited, vector<int> &pi)
38 {
39     if (visited[u])
40     {
41         return;
42     }
43     visited[u] = true;
44     for (auto v : G[u])
45     {
46         if (!(visited[v]))
47         {
48             pi[v] = u;
49             dfs(G, v, visited, pi);
50         }
51     }
52 }
53
54 bool dfs_principal(vector<vector<int>> &G, int n)
55 {
56     vector<bool> visited;
57     vector<int> pi;
58     visited.assign(n, false);
59     pi.assign(n, -2);
60     int bairros = 0;
61     for (int u = 0; u < n; u++)
62     {
63         if (!(visited[u]))
64         {
65             bairros++;
66             if (bairros > 1)
```

```
67         {
68             return true;
69         }
70         pi[u] = -1;
71         dfs(G, u, visited, pi);
72     }
73 }
74 return false;
75 }
76
77 bool pontes(vector<vector<int>> &G, int n)
78 {
79     int u, v;
80     vector<vector<int>> Grafo;
81     Grafo = G;
82     for (int u = 0; u < n; u++)
83     {
84         for (auto v : Grafo[u])
85         {
86             apaga_aresta(Grafo, u, v);
87             if (dfs_principal(Grafo, n))
88             {
89                 return true;
90             }
91             Grafo = G;
92         }
93     }
94     return false;
95 }
96
97 int main()
98 {
99     vector<vector<int>> Grafo;
100     int n, m;
101     cin >> n >> m;
102     Grafo.assign(n, vector<int>());
103     leituraGrafo(Grafo, m);
104     if (pontes(Grafo, n))
105     {
106         cout << "S" << endl;
107     }
108     else
109     {
110         cout << "N" << endl;
111     }
112     return 0;
113 }
```

Problema G

Marlóvsky e a Academia

Por Márcio Belo-Filho

Timelimit: 1

O professor Marlóvsky gosta muito de malhar na academia e sempre executar um treino diferente. A academia que ele malha possui uma lista com N equipamentos, cada equipamento com o seu respectivo número de 1 a N . Marlóvsky quer malhar em P equipamentos por dia, mas prefere que as numerações dos equipamentos escolhidos para um dia tenham uma diferença de pelo menos R unidades entre si. O professor Marlóvsky quer saber quantos treinos ele pode montar com essa estratégia.

Entrada O arquivo de entrada possui uma única linha com três inteiros N , P e R , ($1 \leq N \leq 10^3$), ($1 \leq P \leq 10^2$) e ($1 \leq R \leq 10^2$) indicando: o número de equipamentos da academia, o número de equipamentos utilizados em um treino e a diferença mínima entre os números dos equipamentos.

Saída Seu programa deve imprimir uma única linha na saída, contendo o número de treinos possíveis. Como os números podem ser grandes, imprima a resposta módulo 100000007.

Exemplos de Entrada	Exemplos de Saída
8 3 2	20
8 2 3	15
5 5 1	1
1 2 1	0

Table 11. Exemplos de entradas e saídas


```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long comb(long long n, long long p){
5      p = min(p, n - p);
6      if(p < 0) return 0;
7      if(p == 0) return 1;
8      long long res = n;
9      for(long long i = 1; i < p; i++){
10         res = res * (n - i) / (i + 1);
11     }
12     return res;
13 }
14
15 long long comb_mod(long long n, long long p, long long m){
16     p = min(p, n - p);
17     if(p < 0) return 0;
18     if(p == 0) return 1;
19     long long res = n % m;
20     for(long long i = 1; i < p; i++){
21         res = (res * (n - i) / (i + 1)) % m;
22     }
23     return res;
24 }
25
26 long long kaplansky(long long n, long long p, long long r){
27     return comb(n - (p - 1) * r, p);
28 }
29
30 long long kaplansky_mod(long long n, long long p, long long r, long long m){
31     return comb_mod(n - (p - 1) * r, p, m);
32 }
33
34 int main(){
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     long long n, p, r;
39     cin >> n >> p >> r;
40
41     cout << kaplansky_mod(n, p, r - 1, 100000007LL) << "\n";
42
43     return 0;
44 }
```

Problema H

Rosa dos Ventos

Por Heverton Barros de Macêdo

Timelimit: 1

Em uma competição de parapente há um série de provas, sendo a prova de navegação a que demanda maior estratégia por parte da equipe. Na prova de navegação, alguns minutos antes de seu início, os pilotos recebem um roteiro com os pontos a serem sobrevoados. Ganha a prova, o piloto que conseguir passar sobre a maior quantidade de pontos dentro do limite de tempo previamente estabelecido pela direção de prova.

Nesse ano a organização do Campeonato Goiano de Parapente (CGP) decidiu inovar exigindo que todos os pilotos façam somente curvas de 90 graus. Além disso, ao término da prova, antes de pousar, o piloto deve estar voando na mesma direção em que saiu da rampa.

O problema agora é que a direção de prova está com dificuldades para analisar os relatórios que contém as curvas realizadas pelos pilotos.

Ajude a CGP a avaliar o desempenho dos pilotos. Sua missão é construir um programa que possa analisar a sequência de curvas realizadas por um piloto e apresentar a direção em que ele terminou a prova.

Entrada

A entrada contém vários casos de teste. A primeira linha de cada caso corresponde a quantidade de comandos (Q) realizada pelo piloto ($1 \leq Q \leq 100$), assim como a indicação da posição P (Norte, Sul, Leste ou Oeste) da rampa em que ele decolou ($P \in \{N, S, L, O\}$), sendo essas duas informações separadas por um único espaço. A segunda linha indica a sequência de comandos (C) realizada pelo piloto, podendo ser D , E ou F , com significado de curva à Direita, curva à Esquerda e siga em Frente, respectivamente ($C \in \{D, E, F\}$).

Saída

O programa deve produzir como saída uma única letra indicando a direção em que o piloto finalizou a prova de navegação, podendo ser N (Norte), S (Sul), L (Leste) ou O (Oeste).

O final da entrada é indicado por $Q = 0$.

Exemplos de Entrada	Exemplos de Saída
1 N E 3 O DDF 7 S DDFDEDD 7 N DFDFEDF 13 O FEFEFEFEFEFDD 0	O L S S N

Table 12. Exemplos de entradas e saídas

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  char conv_Letra(int p){
5      if(p == 0) return 'N';
6      if(p == 1) return 'L';
7      if(p == 2) return 'S';
8      if(p == 3) return 'O';
9      exit(0);
10 }
11
12 int conv_Num(char dir_ini){
13     if(dir_ini == 'N') return 0;
14     if(dir_ini == 'L') return 1;
15     if(dir_ini == 'S') return 2;
16     if(dir_ini == 'O') return 3;
17     exit(0);
18 }
19
20 char calc_dir(const string &c, int qtd_com, char dir_ini){
21     int qtd_D = 0, qtd_E = 0;
22
23     for(int i = 0; i < qtd_com; i++){
24         if(c[i] == 'D') qtd_D++;
25         if(c[i] == 'E') qtd_E++;
26     }
27
28     int vlr = qtd_D - qtd_E;
29     int vlr_ini = conv_Num(dir_ini);
30     int tmp = (vlr_ini + vlr) % 4;
31
32     if(tmp >= 0) return conv_Letra(tmp);
33     return conv_Letra(4 + tmp);
34 }
35
36 int main(){
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     int qtd_com;
41     char dir_ini;
42
43     while(true){
44         if(!(cin >> qtd_com)) break;
45         if(qtd_com <= 0) break;

```

```
47         cin >> dir_ini;
48
49         string c;
50         c.reserve(qtd_com);
51         for(int i = 0; i < qtd_com; i++){
52             char x;
53             cin >> x;
54             c.push_back(x);
55         }
56
57         cout << calc_dir(c, qtd_com, dir_ini) << "\n";
58     }
59
60     return 0;
61 }
```

Problema I

Campo de Abóboras

Por Athos José

Timelimit: 2

Todos os anos varias pessoas participam do Abóbora Contest e neste ano tivemos o ilustre e novo participante Athos. Como principiante, ele plantou várias covas de abóboras sem saber ao certo quantas covas era necessário para ter uma quantidade razoável de abóboras para a competição.

Olhando para o campo por cima, vemos que ele está disposto em um formato de matriz $N \times M$, onde cada cova seria uma posição da matriz que possui uma quantidade de abóboras.

Dito isso, Athos pediu para você que responde-se Q perguntas para ele, as pergunta é dizer quantas abóboras há de um ponto A ao ponto B, sendo A e B posições do campo, ambos inclusos.

Por exemplo, dados os campos em disposição de uma matriz 5×6 , e os pontos A e B respectivamente sendo $(2, 2)$ e $(3, 5)$, a pergunta é dizer quantas abóboras haveria na região cinza.

	1	2	3	4	5	6
1						
2						
3						
4						
5						

Entrada

A primeira linha contém três números inteiros N , M e Q ($1 \leq N, M \leq 10^3$ e $1 \leq Q \leq 3 * 10^3$), indicando respectivamente de linhas da matriz, colunas da matriz e o número de perguntas.

Às próximas N linhas conterà M valores $C_{i,j}$ ($1 \leq C_{i,j} \leq 5 \mid 1 \leq i \leq N$ e $1 \leq j \leq M$) que é a quantidade de abóbora em cada cova.

Às próximas Q linhas contem 4 números inteiros que são às coordenadas do A e B ($1 \leq A_x \leq B_x \leq N$) e ($1 \leq A_y \leq B_y \leq M$).

Saída

A saída deve conter apenas números para cada pergunta feita sendo ele a quantidade de abóboras de A à B.

Exemplos de Entrada	Exemplos de Saída
5 6 4	25
1 5 4 3 2 1	45
1 2 3 4 5 2	28
2 1 4 3 3 5	90
4 1 5 2 4 3	5
4 3 2 5 5 1	
2 2 3 5	
1 1 4 4	
2 3 5 4	
1 1 5 6	
2 5 2 5	

Table 13. Exemplos de entradas e saídas

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  typedef vector<vector<int>> vvi;
6  int covas[1000][1000];
7
8  void processo(vvi &dp, int n, int m){
9      for(int i = 1; i <= n; i++){
10         dp[i][1] = covas[i-1][0]+dp[i-1][1];
11         for(int j = 1; j <= m; j++){
12             dp[1][j] = covas[0][j-1]+dp[1][j];
13             for(int i = 1; i <= n; i++){
14                 for(int j = 1; j <= m; j++){
15                     dp[i][j] = dp[i-1][j]+dp[i][j-1]-dp[i-1][j-1]+covas[i-1][j-1];
16                 }
17             }
18         }
19     }
20
21     return;
22 }
23
24 int solv(vvi &dp, int a, int b, int c, int d){
25     return dp[c][d]-dp[a-1][d]-dp[c][b-1]+dp[a-1][b-1];
26 }
27
28 void leitura(int n, int m){
29     for(int i = 0; i < n; i++){
30         for (int j = 0; j < m; j++){
31             cin >> covas[i][j];
32         }
33     }
34     return;
35 }
36
37 int main(){
38     int n, m, q;
39     int a, b, c, d;
40     cin >> n >> m >> q;
41     vvi dp(n+1, vector<int>(m+1, 0));
42
43     leitura(n, m);
44     processo(dp, n, m);
45     while(q--){
46         cin >> a >> b >> c >> d;
47         cout << solv(dp, a, b, c, d) << endl;
48     }
49     return 0;

```


Problema J

Álbum da Copa

Por OBI BR Brasil

Timelimit: 1

Em ano de Copa do Mundo de Futebol, o álbum de figurinhas oficial é sempre um grande sucesso entre crianças e também entre adultos. Para quem não conhece, o álbum contém espaços numerados de 1 a N para colar as figurinhas; cada figurinha, também numerada de 1 a N , é uma pequena foto de um jogador de uma das seleções que jogará a Copa do Mundo. O objetivo é colar todas as figurinhas nos respectivos espaços no álbum, de modo a completar o álbum (ou seja, não deixar nenhum espaço sem a correspondente figurinha).

As figurinhas são vendidas em envelopes fechados, de forma que o comprador não sabe quais figurinhas está comprando, e pode ocorrer de comprar uma figurinha que ele já tenha colado no álbum.

Para ajudar os usuários, a empresa responsável pela venda do álbum e das figurinhas quer criar um aplicativo que permita gerenciar facilmente as figurinhas que faltam para completar o álbum e está solicitando a sua ajuda.

Dados o número total de espaços e figurinhas do álbum, e uma lista das figurinhas já compradas (que pode conter figurinhas repetidas), sua tarefa é determinar quantas figurinhas faltam para completar o álbum.

Entrada A primeira linha contém um inteiro N ($1 \leq N \leq 100$) indicando o número total de figurinhas e espaços no álbum. A segunda linha contém um inteiro M ($1 \leq M \leq 300$) indicando o número de figurinhas já compradas. Cada uma das M linhas seguintes contém um número inteiro X ($1 \leq X \leq N$) indicando uma figurinha já comprada.

Saída Seu programa deve produzir uma única linha contendo um inteiro representando o número de figurinhas que falta para completar o álbum.

Exemplos de Entrada	Exemplos de Saída
10 3 5 8 3	7
5 6 3 3 2 3 3 3	3
3 4 2 1 3 3	0

Table 14. OBI - Olimpíada Brasileira de Informática 2018 - Fase 1


```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int leitura() {
5      int num;
6      cin >> num;
7      return num;
8  }
9
10 void zerar(int n, vector<int> &vet) {
11     for(int i = 0; i < n; i++) {
12         vet[i] = 0;
13     }
14 }
15
16 void ler_vetor(int m, vector<int> &vet) {
17     for(int i = 0; i < m; i++) {
18         int x = leitura();
19         vet[x - 1]++;
20     }
21 }
22
23 int total(int n, vector<int> &vet) {
24     int c = 0;
25     for(int i = 0; i < n; i++) {
26         if(vet[i] == 0) {
27             c++;
28         }
29     }
30     return c;
31 }
32
33 int main() {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36
37     int n = leitura();
38     int m = leitura();
39
40     vector<int> vet(n);
41     zerar(n, vet);
42     ler_vetor(m, vet);
43
44     cout << total(n, vet) << "\n";
45
46     return 0;
47 }
```

Informações Gerais

Este caderno contém 10 problemas; as páginas estão numeradas de 1 a 16, contando esta página de rosto. Verifique se o caderno está completo.

1. Sobre os nomes dos programas

- (a) Para soluções em C/C++ e Python 3.8, o nome do arquivo-fonte não é significativo, pode ser qualquer nome desde que tenha as extensões .c, .cc, .py3.
- (b) Para linguagem Java, sua solução deve ser chamado de `codigo_do_problema.java`, em que `codigo_do_problema` é a letra maiúscula que identifica o problema. Lembre que em Java o nome da classe principal deve ser igual ao nome do arquivo.

2. Sobre a entrada

- (a) A entrada de seu programa deve ser lida da entrada padrão.
- (b) A entrada é composta de um único caso de teste, descrito em um número de linhas que depende do problema.
- (c) Quando uma linha da entrada contém vários valores, estes são separados por um único espaço em branco; a entrada não contém nenhum outro espaço em branco.
- (d) Cada linha, incluindo a última, contém exatamente um caractere final-de-linha.
- (e) O final da entrada coincide com o final do arquivo.

3. Sobre a saída

- (a) A saída de seu programa deve ser escrita na saída padrão.
- (b) Quando uma linha da saída contém vários valores, estes devem ser separados por um único espaço em branco; a saída não deve conter nenhum outro espaço em branco.
- (c) Cada linha, incluindo a última, deve conter exatamente um caractere final-de-linha.

Maratona de Programação Abóbora Contest 2023

Problema A

A liga dos ex-namorados

Por Athos José

Timelimit: 1

No mundo caótico e surreal de “Scott Pilgrim Contra o Mundo”, Scott Pilgrim deve enfrentar os sete ex-namorados malignos da misteriosa Ramona Flowers para ganhar o coração dela. Você é desafiado a escrever um programa que, dado um número entre 1 e 7, exiba o nome do ex-namorado correspondente que Scott Pilgrim deve enfrentar.

Aqui estão os nomes dos ex-namorados e seus respectivos números:

1. Matthew Patel
2. Lucas Lee
3. Todd Ingram
4. Roxanne Richter
5. Kyle Katayanagi
6. Ken Katayanagi
7. Gideon Graves

Entrada

A entrada consiste de um único número inteiro N ($1 \leq N \leq 7$), representando o número do ex-namorado que deve ser exibido.

Saída

Seu programa deve imprimir o nome do ex-namorado correspondente ao número N , conforme a lista acima.

Exemplos de Entrada	Exemplos de Saída
7	Gideon Graves
3	Todd Ingram

Table 15. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int n;
7      cin >> n;
8      if(n == 1){
9          cout << "Matthew Patel" << endl;
10     }else if(n == 2){
11         cout << "Lucas Lee" << endl;
12     }else if(n == 3){
13         cout << "Todd Ingram" << endl;
14     }else if(n == 4){
15         cout << "Roxanne Richter" << endl;
16     }else if(n == 5){
17         cout << "Kyle Katayanagi" << endl;
18     }else if(n == 6){
19         cout << "Ken Katayanagi" << endl;
20     }else if(n == 7){
21         cout << "Gideon Graves" << endl;
22     }
23     return;
24 }
```

Problema B

Brilho Cósmico

Por Athos José

Timelimit: 1

Na exploração fascinante do vasto universo, a medida das distâncias entre os corpos celestes e a apreciação de sua luminosidade, expressa através da magnitude, desempenham papéis cruciais. A distância é frequentemente quantificada em anos-luz, representando o tempo que a luz levaria para viajar da Terra à estrela em questão. A magnitude, por outro lado, é expressa em uma escala logarítmica que denota a luminosidade aparente de uma estrela. Nesta escala, quanto menor é o valor, mais luminosa é a estrela.

A ordenação das estrelas é uma tarefa crucial para os astrônomos e entusiastas do espaço, onde muitas das vezes é dada pela a magnitude aparente de uma estrela. Na Figura 1, podemos visualizar a estrela "Antares", conhecida como a principal estrela da constelação de Escorpião. Antares possui uma magnitude aparente de 0.91, sendo a estrela com a menor magnitude da constelação. Tal característica se reflete em sua notável visibilidade entre outras estrelas.

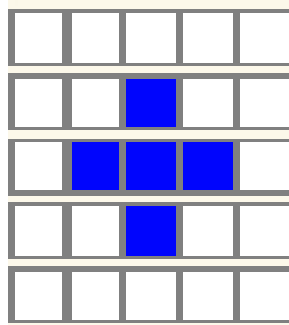


Figure 2. Constelação de Escorpião. Autor: Athos

Na mesma figura temos as estrelas "EpsilonScorpii" e "DeltaScorpii" que possuem uma magnitude aparente de 2,29. Para esses caso é comum utilizar a distância, onde, elas respectivamente possuem 651y e 4021y (sendo ly light-year ou ano-luz).

Nesta emocionante jornada cósmica, você é desafiado a criar um programa que organize as estrelas. Cada estrela é representada por um nome, uma distância em anos-luz e uma magnitude.

A tarefa é ordenar as estrelas em ordem crescente de magnitude aparente, levando em conta tanto os valores positivos quanto negativos. Em caso de empate nas magnitudes, as estrelas devem ser ordenadas em ordem decrescente pela a distância em relação à terra.

Entrada

A entrada consiste de um número inteiro N ($1 \leq N \leq 10^5$), representando o número de estrelas. Em seguida, seguem N linhas, cada uma contendo a descrição de uma estrela no formato "Nome Distancia Magnitude", onde "Nome" é o nome da estrela (uma string sem espaços e com no máximo 50 caracteres), "Distancia" que é uma número inteiro representando a distância da estrela em anos-luz ($1 \leq Distancia \leq 10^9$) e "Magnitude" é um número decimal representando a magnitude da estrela ($-40.0 \leq Magnitude \leq 40.0$).

Saída

Seu programa deve imprimir os nomes da lista de estrelas ordenadas de acordo com as regras estabelecidas.

Exemplos de Entrada	Exemplos de Saída
6 KappaScorpii 464 2.39 EpsilonScorpii 65 2.29 LambdaScorpii 703 1.62 ThetaScorpii 272 1.86 Antares 553 0.91 DeltaScorpii 401 2.29	Antares LambdaScorpii ThetaScorpii DeltaScorpii EpsilonScorpii KappaScorpii

Table 16. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  bool compare(pair<string, pair<double, double>> A, pair<string, pair<double, double
    >> B) {
6      if(A.second.second == B.second.second)
7          return A.second.first > B.second.first;
8      return A.second.second < B.second.second;
9  }
10
11 int main() {
12     vector<pair<string, pair<double, double>>> estrelas;
13     string estrela;
14     double dist, mag;
15     int n;
16     cin >> n;
17     for(int i = 0; i < n; i++){
18         cin >> estrela >> dist >> mag;
19         estrelas.push_back({estrela, {dist, mag}});
20     }
21     sort(estrelas.begin(), estrelas.end(), compare);
22     for(auto x : estrelas){
23         cout << x.first << endl;
24     }
25     return 0;
26 }
```

Problema C

Abobonagramator

Por Rafael Castro

Timelimit: 1

Após anos de intenso desenvolvimento, os cientistas do instituto *edrev oir gfi* finalmente lançaram sua invenção que promete ser revolucionária: O Abobonagramator. Essa máquina é capaz de transformar qualquer palavra com pelo menos 7 letras, em um anagrama de "abobora". Uma palavra é anagrama de outra quando podemos reordenar as letras da primeira até virar a segunda, por exemplo "edrev" e "verde" são anagramas, assim como "oir" e "rio".

A máquina funciona realizando duas operações básicas, sobre qualquer palavra T , sucessivamente:

1. Substituição de uma letra por outra; ou seja, escolhendo uma letra c e um índice i e atribuindo o valor $T_i = c$. Por exemplo, se $T = \text{"abacate"}$ e for escolhida a letra em negrito para ser substituída por 'e', então a nova palavra será "ebacate". Essa operação custa 5 unidades de bateria da máquina.
2. Exclusão de uma letra; ou seja, escolhendo um índice i , e excluindo a letra T_i , deslocando as outras letras a frente de i uma posição. Por exemplo, se $T = \text{"uvauvauva"}$ e for escolhida a letra em negrito para ser excluída, então a nova palavra será "uauvauva". Essa operação custa 3 unidades de bateria da máquina.

Dessa forma, calcule qual a mínima quantidade de unidades de bateria que o Abobonagramator precisa gastar para transformar uma palavra S qualquer em um anagrama da palavra "abobora".

Entrada

A entrada consistirá de uma única linha contendo uma palavra S , contendo letras do alfabeto (a..z), minúsculas e sem acento.

- $7 \leq |S| \leq 10^5$;

Saída

A saída deve conter um único inteiro: A menor quantidade de unidades de bateria que a máquina gasta para transforma S em um anagrama de "abobora". É garantido que sempre existe uma forma de transformar a palavra em um anagrama de "abobora".

Exemplos de Entrada	Exemplos de Saída
abbobora	3
baoborex	8
uvauvauva	28

Table 17. Exemplos de entradas e saídas


```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      string s;
9      cin >> s;
10
11     unordered_map<char, int> abobora_cnt;
12     abobora_cnt['a'] = 2;
13     abobora_cnt['b'] = 2;
14     abobora_cnt['o'] = 2;
15     abobora_cnt['r'] = 1;
16
17     int remove_cost = 3;
18     int substitution_cost = 5;
19
20     long long cost = 0;
21
22     for(auto &p : abobora_cnt){
23         char x = p.first;
24         int need = p.second;
25         int cnt = 0;
26         for(char y : s){
27             if(y == x) cnt++;
28         }
29         if(cnt < need){
30             cost += (long long)substitution_cost * (need - cnt);
31         }
32     }
33
34     cost += (long long)remove_cost * ((int)s.size() - 7);
35
36     cout << cost << "\n";
37
38     return 0;
39 }
```

Problema D

Colmeia

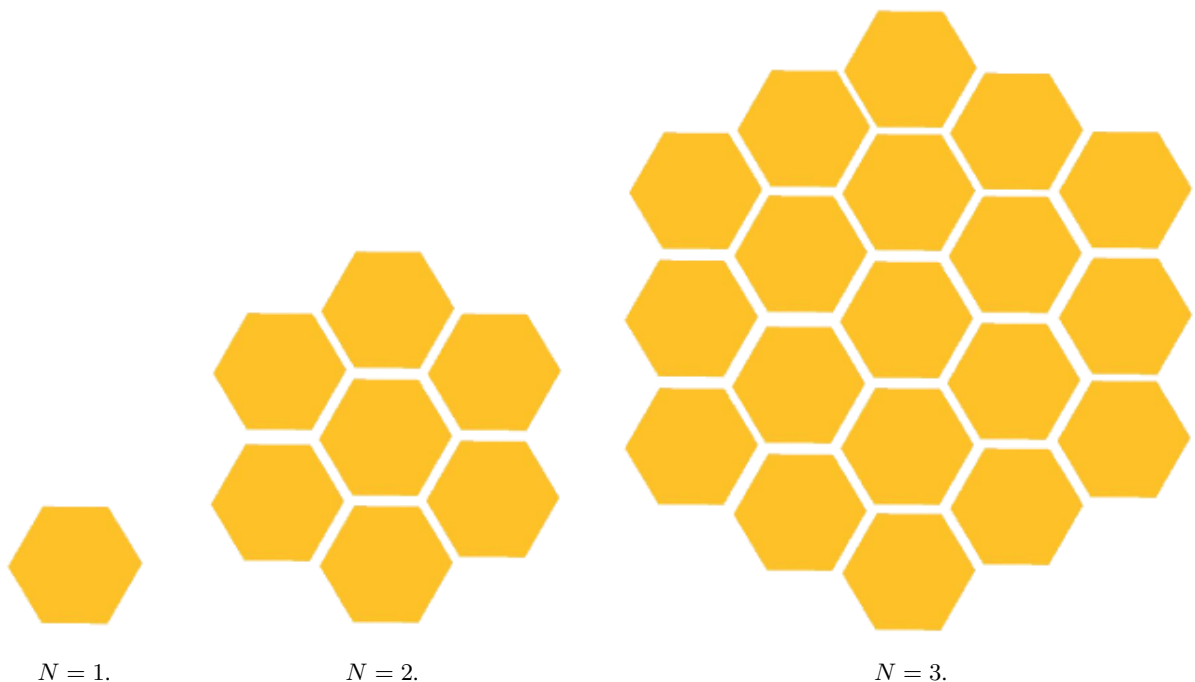
Por Gustavo Leal

Timelimit: 1

Dayllon vem de uma tradicional família de fazendeiros. Há séculos sua família planta diversos tipos de plantas da família Cucurbitaceae, de cabotiás a abobrinhas, todas são parte da produção familiar. Mas infelizmente as plantações do ultimo ano não foram produtivas, pois a região está com falta de abelhas, impedindo que qualquer tipo de legume prospere.

Muito Perspicaz, Dayllon decidiu criar abelhas, para assim mitigar esta crise. Ele escolheu criar um tipo específico de abelhas, nomeado-a de abelius padronos. As abelhas deste tipo seguem padrões extremamente específicos. A sua colmeia, por exemplo, é composta por alvéolos hexagonais como qualquer outra, mas eles seguem uma padrão muito bem definido. Toda colmeia pode ser definida através de uma profundidade N .

As abelhas começam a construir as colmeias a partir de um alvéolo central. Depois disso constroem alvéolos adjacentes aos alvéolos que estão na borda da colmeia, e assim por diante, até que tenham feito isso $N - 1$ vezes. Seguem alguns exemplos:



Depois de construir as colmeias, as abelhas começam a carregar bolhas de pólen para os alvéolos. Cada abelha consegue preencher um alvéolo por completo em uma viagem, mas elas são muito restritivas. Elas só colocarão pólen em um alvéolo se 3 ou mais alvéolos adjacentes já estiverem cheios de pólen.

Dayllon percebeu que isso implica que alguns alvéolos não serão preenchidos pelas abelhas, indicando que elas irão trabalhar menos. Ele então decidiu preencher alguns alvéolos manualmente, de forma que todos os alvéolos da colmeia sejam preenchidos por pólen no final. Ajude Dayllon a descobrir qual é o mínimo de alvéolos que ele precisa preencher, para uma colmeia com profundidade N .

Entrada

A entrada consiste de uma linha com um inteiro N representando a profundidade da colmeia.

- $1 \leq N \leq 10^9$;

Saída

Imprima um inteiro representando a quantidade de alvéolos que o Dayllon precisa de preencher.

Exemplos de Entrada	Exemplos de Saída
1	1
2	3

Table 18. Exemplos de entradas e saídas

Notas

Para o exemplo 1 as abelhas nunca irão preencher uma colmeia com somente um alvéolo, então a única possibilidade seria Dayllon preenche-lo manualmente.

Para o exemplo 2, o Dayllon pode seguir os passos a seguir:



Neste caso as abelhas irão preencher o alvéolo central, pois ele possui 3 alvéolos adjacentes preenchidos. Depois disso, todos os alvéolos restantes também terão 3 alvéolos adjacentes preenchidos.

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4
5  int main()
6  {
7      long long n;
8      cin >> n;
9      cout << 2*(n-1) + 1 << '\n';
10     return 0;
11 }
```

Problema E

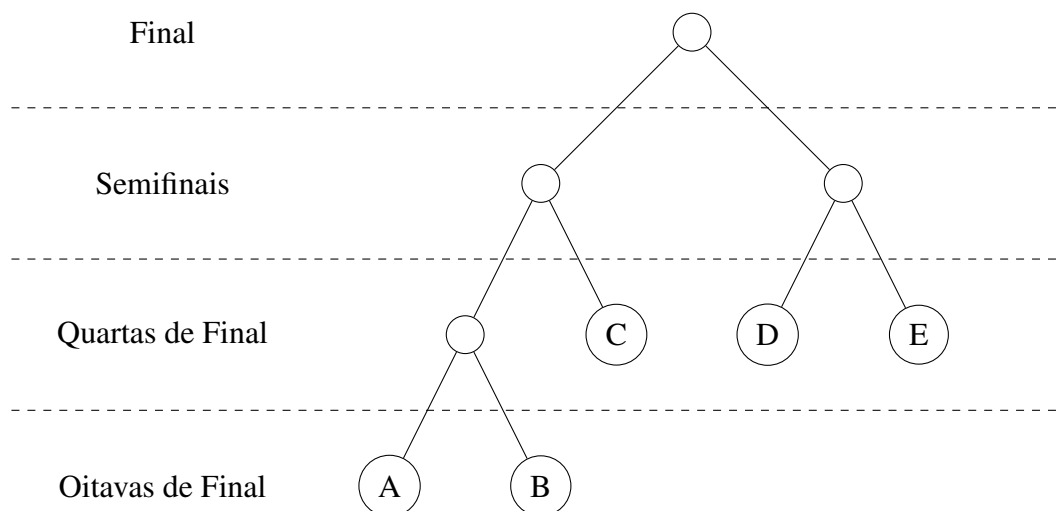
Encontro na Final

Por Márcio Belo

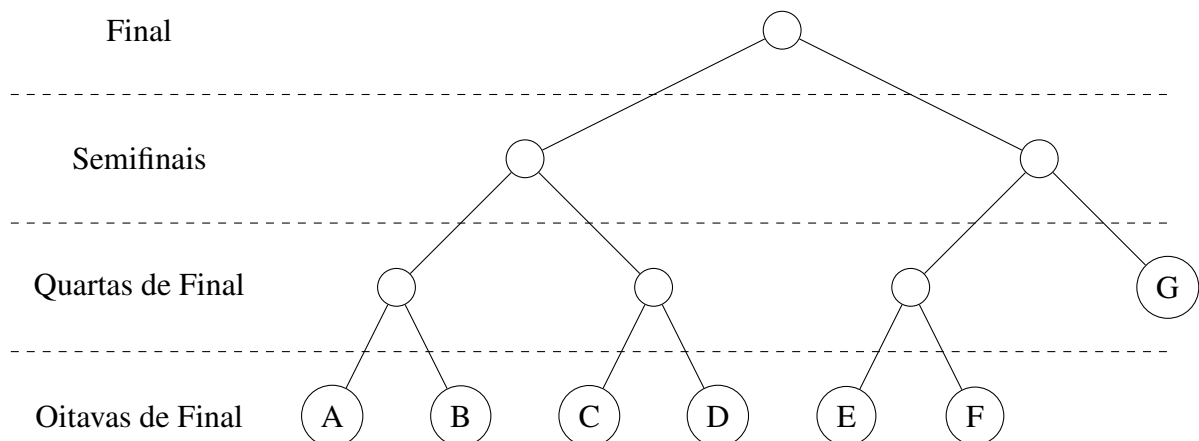
Timelimit: 1

No torneio esportivo Copa do Belo, as partidas são disputadas por dois participantes e são do tipo mata-mata, ou seja, o ganhador segue para a próxima fase, e o perdedor é eliminado da competição. O chaveamento do torneio é fixo e os participantes vencedores vão avançando nas chaves conhecidas até que haja dois times na final. O chaveamento do torneio é uma árvore binária de jogos onde as folhas são os times da competição. Os nós pais da árvore serão ocupados pelo time vencedor dentre os dois nós filhos. Entretanto, o número de participantes iniciais do torneio não necessariamente é uma potência de 2, por isso, no chaveamento, alguns times podem estar em uma fase anterior no início do torneio. Nesse caso, considere sempre o chaveamento em que a árvore está balanceada para a esquerda. Além disso, o chaveamento dos times é feito de forma aleatória para os nós folhas da árvore.

O exemplo a seguir apresenta um chaveamento para $n = 5$ times. Como não é potência de 2, o chaveamento fica desbalanceado para esquerda.



Para um chaveamento $n = 7$ times, o chaveamento fica da seguinte forma



O prof. Belo sabe que dois times são muito fortes e vão ganhar todos os seus jogos, e um encontro entre eles é esperado. A questão é, qual a probabilidade desses dois times se encontrarem na final da Copa do Belo?

Entrada Cada entrada contém uma única linha com o número N de times, com $(2 \leq N \leq 10^9)$.

Saída O sistema deve retornar a probabilidade de dois times que vencem todos os jogos se encontrarem na final. A probabilidade deve estar em porcentagem, com 3 casas decimais. Não é necessário imprimir o símbolo "%".

Exemplos de Entrada	Exemplos de Saída
2	100.000
3	66.667
8	57.143
500	50.071

Table 19. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  double encontronafinal(long long n){
5      long long k = 0;
6      while((1LL << (k + 1)) <= n) k++;
7
8      long long extra = n - (1LL << k);
9      long long a;
10
11     if(extra <= (1LL << (k - 1))){
12         a = 2LL * ((1LL << (k - 1)) + extra) * (1LL << (k - 1));
13     }else{
14         extra -= (1LL << (k - 1));
15         a = 2LL * (1LL << k) * ((1LL << (k - 1)) + extra);
16     }
17
18     long long b = n * (n - 1);
19     return 100.0 * (double)a / (double)b;
20 }
21
22 int main(){
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     long long n;
27     cin >> n;
28
29     double sol = encontronafinal(n);
30     cout << fixed << setprecision(3) << sol << "\n";
31
32     return 0;
33 }
```

Problema F

Quero imprimir em 3D!

Por Douglas Cedrim

Timelimit: 1

Impressão 3D é uma tecnologia arretada, que está em alta, e que possibilita imprimir uma peça tridimensional utilizando: um modelo digital devidamente processado e um *filamento* plástico - que irá constituir o material da peça impressa. A ideia geral é que um modelo digital (modelo original) seja convertido em uma versão do modelo fatiado ao longo do eixo Z, e a impressora 3D irá depositar filamento, de baixo para cima, uma fatia por vez, uma sobre a outra, ao longo de planos paralelos ao plano XY, seguindo o caminho dos contornos do objeto e de seu interior, que é definido pelo *padrão de preenchimento*.

Na Figura 3a esse processo é ilustrado de forma que cada fatia (h) está representada em vermelho, com um padrão de preenchimento (quadriláteros) representado em laranja. Note que outros padrões de preenchimento podem ser usados. Assim, o total de filamento na fatia corresponde à combinação desse caminho formado pelas cores vermelha e laranja. Na Figura 3b, por exemplo, é ilustrado todo o sistema mostrando o filamento e a peça impressa, com um total de 6 fatias ($h = 6$).

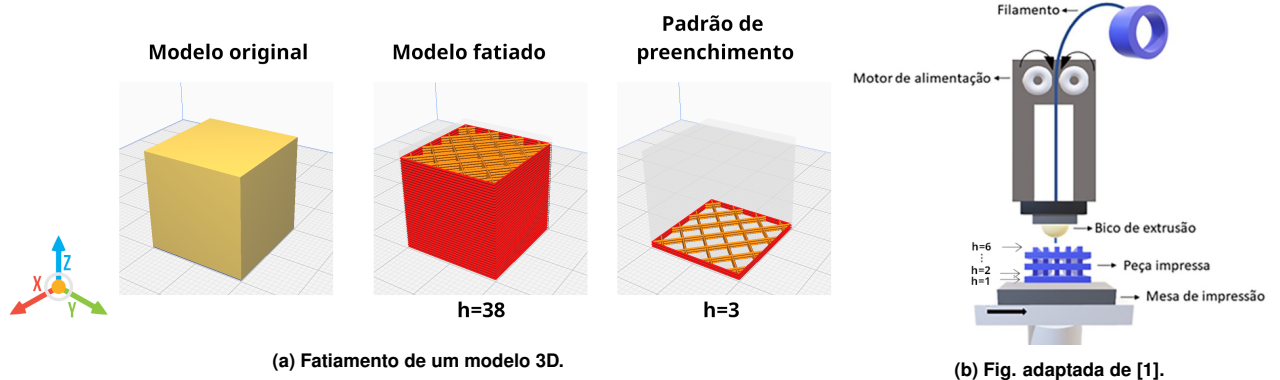


Figure 3. Impressão 3D.

O professor Márcio Belo recebeu alguns modelos digitais já fatiados, e com um número fixo de fatias diferente para cada modelo (h_{max}). Ele precisa calcular o quanto de filamento ele irá necessitar para efetuar a impressão de cada modelo. Para ajudá-lo você deverá estimar o quanto precisará de filamento em cada fatia h e então calcular o montante para o modelo inteiro. Para simplificar, dado um modelo M , todas as suas fatias são iguais e vão consumir a mesma quantidade de filamento. Além disso, considere também que o montante de filamento que pode ser depositado entre fatias subsequentes é irrisório, irrelevante para sua estimativa. O comprimento de filamento consumido por unidade de comprimento de impressão é igual.

Para um modelo M você terá disponível um conjunto de vértices $V = \{v_i\}_{i=1}^n$, ordenados no sentido anti-horário, com coordenadas (x_i, y_i) que definem o contorno externo da fatia como um polígono convexo. O seu padrão de preenchimento deverá ser triangular, análogo à triangulação $\mathcal{T}$ ilustrada em vermelho na Figura 4, de forma que sempre o primeiro vértice v_1 é comum a todos os triângulos. Assim, os vértices de qualquer triângulo $t \in \mathcal{T}$ estarão contidos em $\{v_i\}$.

Para fins de otimização do uso do filamento, a impressora está configurada para não passar pela mesma aresta mais de uma vez.

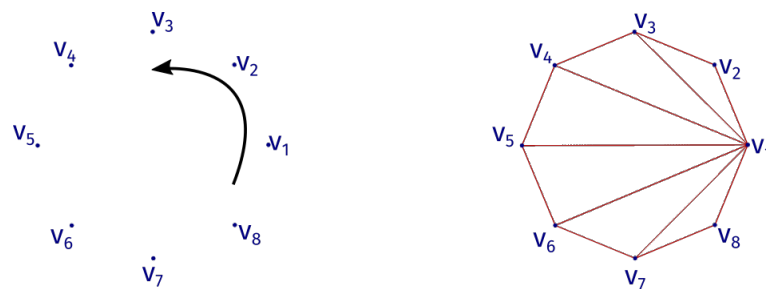


Figure 4. Triangulação $\mathcal{T}$ de um modelo M .

[1] BERNARDO, Marcela Piassi et al. *Processamento e aplicação de biomateriais poliméricos: avanços recentes e perspectivas*. Química Nova, v. 44, p. 1311-1327, 2021.

Entrada Cada entrada contém os dados do modelo M fatiado. A primeira linha contém a quantidade de fatias do modelo h_{max} . A segunda linha contém o número de vértices n , com $(3 \leq n \leq 10^2)$. A terceira linha é formada pela sequência de vértices na forma de suas coordenadas x_i, y_i , com cada coordenada representada em ponto flutuante, com precisão de três casas decimais.

Saída O sistema deve retornar o montante total de filamento necessário para impressão do modelo M como um número inteiro. Para isso, efetue o arredondamento apenas ao final das contas, para o menor número inteiro maior ou igual que o montante.

Exemplos de Entrada	Exemplos de Saída
16 4 0.0 0.0 4.0 0.0 4.0 1.0 0.0 1.0	226
85 10 1.000 0.000 0.809 0.588 0.309 0.951 -0.309 0.951 -0.809 0.588 -1.000 0.000 -0.809 -0.588 -0.309 -0.951 0.309 -0.951 0.809 -0.588	1494
32 16 3.000 0.000 2.772 0.383 2.121 0.707 1.148 0.924 0.000 1.000 -1.148 0.924 -2.121 0.707 -2.772 0.383 -3.000 0.000 -2.772 -0.383 -2.121 -0.707 -1.148 -0.924 -0.000 -1.000 1.148 -0.924 2.121 -0.707 2.772 -0.383	1998

Table 20. Exemplos de entradas e saídas

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  double dist_l2(const pair<double,double> &v1, const pair<double,double> &v2){
5      double dx = v1.first - v2.first;
6      double dy = v1.second - v2.second;
7      return sqrt(dx * dx + dy * dy);
8  }
9
10 int main(){
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     int h_max;
15     cin >> h_max;
16
17     int n_vertices;
18     cin >> n_vertices;
19
20     vector<pair<double,double>> V(n_vertices);
21     for(int i = 0; i < n_vertices; i++){
22         cin >> V[i].first >> V[i].second;
23     }
24
25     double total = 0.0;
26     for(int i = 1; i < n_vertices - 1; i++){
27         double ei = dist_l2(V[i], V[i + 1]);
28         double ej = dist_l2(V[i + 1], V[0]);
29         total += (ei + ej);
30     }
31     total += dist_l2(V[0], V[1]);
32
33     long long resultado = (long long)ceil(h_max * total);
34     cout << resultado << "\n";
35
36     return 0;
37 }

```

Problema G

Fabiola e os cafés

Por Cleon Xavier Pereira Júnior

Timelimit: 1

Professora Fabiola é fascinada por cafés de qualidade. Em 2022, a professora Fabiola pegou seu Ford Ka e, com sua filha, foi aventurar-se pelo sul de minas para experimentar os melhores cafés do país. Em cada produtora de cafés especiais que Fabiola visitava, ela levava um pacote. Os pacotes tinham diferentes valores e pesos.

Acontece que, na empolgação, a professora Fabiola esqueceu que tinha um Ford Ka, uma filha e umas bagagens. Já sabem o que aconteceu né? Eis que a professora Fabiola tinha um problema de excesso de carga. Ao término da viagem, quando ela colocou tudo no carro, a professora se deparou com um carro rebaixado, mas não era por estilização, era por questão de peso mesmo.

Observando a inviabilidade da situação, a professora Fabiola notou que deveria deixar uns cafés para trás. Claro que ela gostaria de levar o máximo de cafés possíveis e que tivesse o menor prejuízo. Para isso, ela estava disposta, se necessário, a abrir alguns pacotes de café e levar só parte desse, para ter a possibilidade de levar a capacidade máxima.

No momento, a professora Fabiola só está preocupada em saber o tamanho do prejuízo financeiro que ela terá na melhor situação. Ou seja, quanto ela gastou e não conseguirá levar por excesso de peso. Vamos ajudar a professora fazendo este cálculo para ela?

Entrada Cada entrada contém:

- um inteiro X que corresponderá aos M pacotes de cafés;
- um número Y , correspondente à capacidade de carga disponível no carro para os cafés;
- M linhas com dois números, separados por um espaço, sendo o peso, seguido do valor.

Saída O algoritmo deve retornar um valor numérico, com duas casas decimais, correspondendo ao prejuízo financeiro que a professora terá.

Exemplos de Entrada	Exemplos de Saída
5 50 40 840 30 600 20 400 10 100 20 300	1200.00
3 450 420 1500 450 1200 480 1550	2653.12

Table 21. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void insertionSort(vector<double> &array, vector<double> &p, vector<double> &v){
5      int n = array.size();
6      for(int step = 1; step < n; step++){
7          double key = array[step];
8          double key_1 = p[step];
9          double key_2 = v[step];
10         int j = step - 1;
11
12         while(j >= 0 && key < array[j]){
13             array[j + 1] = array[j];
14             p[j + 1] = p[j];
15             v[j + 1] = v[j];
16             j--;
17         }
18
19         array[j + 1] = key;
20         p[j + 1] = key_1;
21         v[j + 1] = key_2;
22     }
23 }
24
25 int main(){
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int quant;
30     cin >> quant;
31
32     double capacidade;
33     cin >> capacidade;
34
35     vector<double> p(quant), v(quant), razao(quant), inc(quant, 0.0);
36
37     for(int i = 0; i < quant; i++){
38         double x, y;
39         cin >> x >> y;
40         p[i] = x;
41         v[i] = y;
42         razao[i] = y / x;
43     }
44
45     insertionSort(razao, p, v);
46
47     double total = 0.0;
48     for(int i = quant - 1; i >= 0; i--){
49         if(p[i] <= capacidade){
50             inc[i] = 1.0;
51             capacidade -= p[i];
52         }else{
53             inc[i] = capacidade / p[i];
54             capacidade = 0.0;
55         }
56         total += inc[i] * v[i];
57     }
58
59     double soma_v = 0.0;
60     for(double val : v) soma_v += val;
61
62     cout << fixed << setprecision(2) << (soma_v - total) << "\n";
63
64     return 0;
65 }
```

Problema H

Genes Significativos

Por Heverton Barros de Macêdo

Timelimit: 1

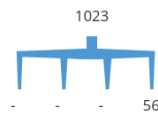
Charles, um famoso pesquisador da evolução das espécies passou a investigar vidas artificiais. Em seus estudos, constatou que para cada amostra genética de um indivíduo da população estudada apenas alguns genes são significativos. A suposição do pesquisador para encontrar os genes significativos é um tanto quanto curiosa e foi realizada da seguinte forma:

Inicialmente foi definido um número para cada gene que pode variar de 0 a 10000. A quantidade de genes para cada indivíduo não é conhecida, mas para fins de estudos foi estipulada como variando de 1 a 100. A partir da sequência genética é montada uma espécie de hierarquia, no formato de um organograma, em que o primeiro gene é o pai. A partir do segundo gene, a inserção no organograma ocorre multiplicando o número do gene pai com o número do gene que está sendo inserido, para então extrair os seus dois últimos dígitos, que definem o chamado valor hierárquico. O valor hierárquico é responsável por indicar a posição no organograma em que esse gene será inserido. Uma restrição na construção do organograma é que cada gene possui 4 filhos, nomeados como filho1, filho2, filho3 e filho4, em que cada filho representa 1/4 do valor hierárquico (filho1 de 0 a 24, filho2 de 25 a 49, filho3 de 50 a 74 e filho4 de 75 a 99).

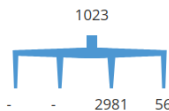
De posse do organograma, a extração dos genes significativos é obtida conforme o nível em que o gene se encontra, mas não se sabe ao certo em qual nível da hierarquia estão os genes significativos.

Para exemplificar, considere a seguinte sequência genética de um indivíduo: 1023 56 2981 517 2 5 8 479 1052 69

A construção do organograma inicia com o primeiro gene 1023 que representa o pai na hierarquia. Em seguida, o gene 56 é multiplicado com o pai (1023) produzindo o número 57288, onde então é obtido o valor hierárquico 88, indicando que o gene 56 será inserido no organograma como filho4 de 1023. Graficamente podemos representar como:

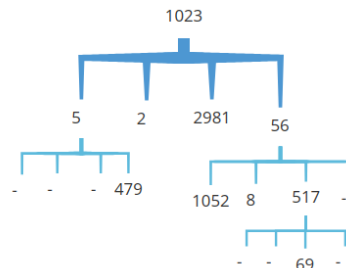


Na sequência, o gene 2981, é inserido no organograma de forma similar, primeiro multiplicando com 1023, produzindo o número 3049563, onde se obtém o valor hierárquico 63, o que indica que será inserido como filho3 de 1023.



O terceiro gene, 517, será inserido como filho3 de 56, visto que inicialmente ocorre a multiplicação de 1023 por 517, produzindo o número 528891, onde se obtém o valor hierárquico 91, indicando que este deveria ser inserido como filho4 de 1023, porém, essa posição já está ocupada, e o processo é então repetido, considerando como pai o gene 56. Multiplicando 56 por 517 será produzido o número 28952 que indica o valor hierárquico 52, ocorrendo assim a inserção do gene 517 como filho3 de 56.

Para a sequência genética do exemplo, podemos obter graficamente o organograma abaixo:



Considerando que os genes significativos do exemplo citado estão no nível 2 do organograma, começando a contagem de níveis em 0, temos como genes significativos: 479, 1052, 8 e 517.

Sua missão é ajudar Charles a obter os genes significativos de um indivíduo e acelerar os estudos desse pesquisador.

Entrada Cada entrada contém os dados de um indivíduo. A primeira linha contém um inteiro X indicando o nível (iniciado em 0) da hierarquia onde estão os genes ($0 \leq X \leq 99$). A segunda linha representa a quantidade N de genes de um indivíduo ($1 \leq N \leq 100$). A terceira linha é formada pela sequência S de genes de um indivíduo, sendo que cada elemento está no intervalo de 0 a 10000 ($0 \leq S[i] \leq 10000, 0 \leq i < N$).

Saída O sistema deve retornar a sequência de genes significativos seguido do caractere #, todos separados por um único espaço, conforme o nível hierárquico X , indicado na primeira linha da entrada. Os filhos 1, 2, 3 e 4, devem ser representados da esquerda para direita. Se nenhum gene significativo for encontrado no nível indicado, a saída terá apenas o caractere #.

Exemplos de Entrada	Exemplos de Saída
0 10 1023 56 2981 517 2 5 8 479 1052 69	1023 #
2 10 1023 56 2981 517 2 5 8 479 1052 69	479 1052 8 517 #
5 10 1023 56 2981 517 2 5 8 479 1052 69	#
2 10 5 98 78 50 7 38 25 61 1579 2501	2501 25 38 78 #

Table 22. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct No{
5      int dado;
6      No *p1, *p2, *p3, *p4;
7  };
8
9  No* criar_no(int dado){
10     No* novo = new No;
11     novo->dado = dado;
12     novo->p1 = nullptr;
13     novo->p2 = nullptr;
14     novo->p3 = nullptr;
15     novo->p4 = nullptr;
16     return novo;
17 }
18
19 int extrair_dois_ultimos_digitos(int vlr){
20     return vlr % 100;
21 }
22
23 void inserir(No* &r, int dado){
24     if(r == nullptr){
25         r = criar_no(dado);
26         return;
27     }
28
29     int vlr = extrair_dois_ultimos_digitos(r->dado * dado);
30
31     if(vlr < 25){
32         inserir(r->p1, dado);
33         return;
34     }
35     if(vlr < 50){
36         inserir(r->p2, dado);
37         return;
38     }
39     if(vlr < 75){
40         inserir(r->p3, dado);
41         return;
42     }
43     inserir(r->p4, dado);
44 }
45
46 void mostrar_nivel(No* r, int nivel){
47     if(r == nullptr) return;
48
49     if(nivel == 0){
50         cout << r->dado << " ";
51     }else{
52         mostrar_nivel(r->p1, nivel - 1);
53         mostrar_nivel(r->p2, nivel - 1);
54         mostrar_nivel(r->p3, nivel - 1);
55         mostrar_nivel(r->p4, nivel - 1);
56     }
57 }
58
59 void destruir_arvore(No* &r){
60     if(r != nullptr){
61         destruir_arvore(r->p1);
62         destruir_arvore(r->p2);
63         destruir_arvore(r->p3);
64         destruir_arvore(r->p4);
65         delete r;
66         r = nullptr;
67     }
```

```
68 }
69
70 int main() {
71     ios::sync_with_stdio(false);
72     cin.tie(nullptr);
73
74     No* r = nullptr;
75     int dado, qtd, nivel;
76
77     if(!(cin >> nivel)) return 0;
78
79     while(true) {
80         if(!(cin >> qtd)) break;
81
82         for(int i = 0; i < qtd; i++) {
83             cin >> dado;
84             inserir(r, dado);
85         }
86
87         mostrar_nivel(r, nivel);
88         cout << "#";
89
90         destruir_arvore(r);
91
92         if(!(cin >> nivel)) break;
93     }
94
95     return 0;
96 }
```

Problema I

Heverton e a Fila

by André da Cunha Ribeiro

Timelimit: 1

Heverton é um coordenador de curso muito dedicado do curso de computação do NuCAT. Os alunos estão enfrentando um grande problema na entrada do NuCAT, todos querem entrar no mesmo horário gerando assim um grande tumulto. Heverton reuniu com os cavalheiros do NDE para tentar resolver o problema. Depois de muita discussão o NDE resolveu ordenar os alunos pelo número de matrícula, e organizando a entrada pela ordem decrescente. Mas Heverton precisa de sua ajuda para desenvolver o programa pois ele está muito ocupado com seus trabalhos.

Entrada Para cada dia será lido um inteiro N , $1 \leq N \leq 10^4$, que representa o número de alunos disponíveis na entrada. Depois será lido os N números de matrículas.

Saída Para cada dia, imprimir os número de matrículas ordenados em ordem decrescente.

Exemplos de Entrada	Exemplos de Saída
5 1 2 3 4 5	5 4 3 2 1

Table 23. Exemplos de entradas e saídas


```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int ler(){
5      int num;
6      cin >> num;
7      return num;
8  }
9
10 void vetor(int n, vector<int> &vet){
11     for(int i = 0; i < n; i++){
12         vet[i] = ler();
13     }
14 }
15
16 void escrever(int n, vector<int> &vet){
17     for(int i = 0; i < n; i++){
18         cout << vet[i] << " ";
19     }
20     cout << "\n";
21 }
22
23 void troca(int &x, int &y){
24     int aux = x;
25     x = y;
26     y = aux;
27 }
28
29 void ordena(int n, vector<int> &vet){
30     for(int i = 0; i < n - 1; i++){
31         int menor = i;
32         for(int j = i; j < n; j++){
33             if(vet[j] > vet[menor]){
34                 menor = j;
35             }
36         }
37         troca(vet[menor], vet[i]);
38     }
39 }
40
41 int main(){
42     ios::sync_with_stdio(false);
43     cin.tie(nullptr);
44
45     int n = ler();
46     vector<int> vet(n);
47     vetor(n, vet);
48     ordena(n, vet);
49     escrever(n, vet);
50
51     return 0;
52 }
```

Problema J

Zé do Grafo

Por André da Cunha Ribeiro

Timelimit: 1

Zé do Grafo, a lenda que ecoa como o Mestre Jedi exterminador de monstros, emerge das sombras como uma faísca ardente de esperança na enigmática cidade entrelaçada nos recônditos do IFGoiano - Campus Rio Verde. Essa cidade, envolta em mistério, desdobra-se perante olhares curiosos como um enigma arquitetônico complexo, com conexões intrincadas que aguçam a mente.

Nesse ambiente intrigante, Zé do Grafo se ergue como um guardião destemido, dotado de poderes transcendentes. Em um dia sereno, enquanto explorava a cidade, ele desvenda um bairro oculto, um santuário de portais secretos que se estendem ao multiverso. Cada portal é adornado com inscrições enigmáticas, revelando as delicadas ligações entre os diversos universos. Uma característica única surge: cada universo conectado é tingido por uma cor distinta, representando a natureza de sua atmosfera. Nota-se que universos vizinhos exibem atmosferas diversas, delineando uma dança cósmica de elementos distintos.

Entretanto, dilemas pairam sobre Zé do Grafo: ele só pode sobreviver em dois tipos específicos de atmosfera, uma limitação impregnada pelas superstições que permeiam a linhagem de mestres Jedi. A narrativa então faz um compasso, direcionando o holofote para um novo protagonista: você. Sua ajuda, inestimável e ancorada na compreensão profunda dos algoritmos e grafos, é convocado para solucionar este enigma vital. A pergunta que se insinua é crucial: há algum portal capaz de permitir que o lendário Zé do Grafo explore os multiversos, sem que isso desafie suas crenças enraizadas na superstição?

Entrada Cada entrada contém os dados de cada portal. A primeira linha contém dois inteiros N, M , indicando respectivamente o número de multiversos ($1 \leq N \leq 50$) e as ligações dos multiversos ($1 \leq M \leq 2000$).

Cada uma das M linhas seguintes composta de dois inteiros que descreve as ligações dos multiverso.

Saída O sistema deve retornar “Sim” caso Zé do Grafos consiga explorar todo o multiverso. Caso contrário, a resposta será “Não”.

Exemplos de Entrada	Exemplos de Saída
6 7 1 2 1 6 2 3 3 4 3 6 4 5 5 6	Sim
6 7 1 2 1 3 2 3 3 4 4 5 4 6 5 6	Não
3 3 1 2 1 3 2 3	Não

Table 24. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  void leituraGrafo(vector<vector<int>> &G, int m)
6  {
7      int a, b;
8      while (m--)
9      {
10         cin >> a >> b;
11         int u = a - 1, v = b - 1;
12
13         G[u].push_back(v);
14         G[v].push_back(u);
15     }
16 }
17
18 bool dfs(vector<vector<int>> &G, int s, vector<int> &cor, int c)
19 {
20     cor[s] = c;
21     for (auto u : G[s])
22     {
23         if (cor[u] == -1)
24         {
25             if (dfs(G, u, cor, 1 - c) == false)
26             {
27                 return false;
28             }
29         }
30         else
31         {
32             if (cor[u] == c)
33             {
34                 return false;
35             }
36         }
37     }
38     return true;
39 }
40
41 int main()
42 {
43     int n, m;
44     cin >> n >> m;
45     vector<vector<int>> Grafo;
46     Grafo.assign(n, vector<int>());
47     leituraGrafo(Grafo, m);
48     vector<bool> visited;
49     visited.assign(n, false);
50     vector<int> cor;
51     cor.assign(n, -1);
52     int u = 0;
53     if (dfs(Grafo, u, cor, 0))
54     {
55         cout << "Sim";
56     }
57     else
58     {
59         cout << "Nao";
60     }
61     cout << endl;
62     return 0;
63 }
```

5º Abóbora Contest

31 de Outubro de 2024

Caderno de Problemas

Informações Gerais

Este caderno contém 8 problemas; as páginas estão numeradas de 1 a 10, contando esta página de rosto. Verifique se o caderno está completo.

1. Sobre os nomes dos programas

- (a) Para soluções em C/C++ e Python 3.10, o nome do arquivo-fonte não é significativo, pode ser qualquer nome desde que tenha as extensões .c, .cpp, .java e .py.
- (b) Para linguagem Java, sua solução deve ser chamado de `codigo_do_problema.java`, em que `codigo_do_problema` é a letra maiúscula que identifica o problema. Lembre que em Java o nome da classe principal deve ser igual ao nome do arquivo.

2. Sobre a entrada

- (a) A entrada de seu programa deve ser lida da entrada padrão.
- (b) A entrada é composta de um único caso de teste, descrito em um número de linhas que depende do problema.
- (c) Quando uma linha da entrada contém vários valores, estes são separados por um único espaço em branco; a entrada não contém nenhum outro espaço em branco.
- (d) Cada linha, incluindo a última, contém exatamente um caractere final-de-linha.
- (e) O final da entrada coincide com o final do arquivo.

3. Sobre a saída

- (a) A saída de seu programa deve ser escrita na saída padrão.
- (b) Quando uma linha da saída contém vários valores, estes devem ser separados por um único espaço em branco; a saída não deve conter nenhum outro espaço em branco.
- (c) Cada linha, incluindo a última, deve conter exatamente um caractere final-de-linha.

Problema A

Abóbora Perfeita

Nome do arquivo: A.c, A.cpp, A.java, ou A.py

Timelimit: 1

Por Athos José

Na cidade de Rio Verde, a produção de abóboras é tão abundante que, ao longo dos anos, surgiram várias lendas sobre abóboras especiais. Uma das lendas mais antigas fala sobre a mítica Abóbora Perfeita.

Segundo a lenda, uma abóbora só pode ser considerada perfeita quando nascem, ao redor da abóbora maior, abóboras menores cujos respectivos tamanhos sejam divisores da abóbora maior. Além disso, a soma dos tamanhos das abóboras menores ao redor deve resultar no tamanho da abóbora maior. Essas propriedades tornam a abóbora perfeita.

Você foi chamado para investigar abóboras de diferentes tamanhos e determinar quais delas podem ser consideradas perfeitas, de acordo com a lenda.

Entrada

A primeira linha contém um inteiro N , representando o número de abóboras ($1 \leq N \leq 10^7$). As N linhas seguintes contém um inteiro T ($1 \leq T \leq 10^4$), onde T indica o tamanho da abóbora.

Saída

Para cada caso de teste imprima “SIM” se a abóbora pode ser considerada perfeita. Caso contrario imprima “NAO”.

Exemplos de Entrada	Exemplos de Saída
4	NAO
9	SIM
6	SIM
28	NAO
21	

Table 25. Exemplos de entradas e saídas

Nota

O exemplo contém 4 casos de testes.

No **primeiro** caso temos $1 + 3 = 4$ que é diferente do tamanho 9.

Já no **segundo** caso temos $1 + 2 + 3 = 6$ que é igual ao tamanho da abóbora 6.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int np[4] = {6, 28, 496, 8128};
5
6  int solver(int x){
7      int ans = 0;
8      for(int i = 0; i < 4; i++){
9          if(x == np[i]){
10             ans = 1;
11         }
12     }
13     return ans;
14 }
15
16 int main(){
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19
20     int n, v;
21     cin >> n;
22     for(int i = 0; i < n; i++){
23         cin >> v;
24         if(solver(v))
25             cout << "SIM\n";
26         else
27             cout << "NAO\n";
28     }
29
30     return 0;
31 }
```

Problema B

Jack Lanterna e as trilhas

Nome do arquivo: B.c, B.cpp, B.java, ou B.py

Timelimit: 1

Por André da Cunha Ribeiro

Jack Lanterna é um caçador de monstros muito temido, que na noite de Halloween decide explorar uma cidade assombrada nos confins de uma floresta. A cidade possui casas mal-assombradas conectadas por trilhas, cada uma com um número de monstros que guardam o caminho.

Cada vez que Jack atravessa uma trilha e derrota os monstros que a guardam, ele destrói a trilha com sua magia, impedindo o retorno por aquele caminho. Jack consegue descansar em cada casa para recuperar suas forças antes de enfrentar mais monstros. Sua missão é derrotar todos os monstros e, ao final, retornar à casa inicial.

Você precisa determinar se Jack Lanterna consegue eliminar todos os monstros e retornar à casa inicial após derrotar todos os monstros.

Entrada A entrada contém vários casos de teste. A primeira linha de cada caso de teste contém dois inteiros N , M , indicando respectivamente o número de casas ($1 \leq N \leq 1000$), e de trilhas ($1 \leq M \leq 1000$).

Cada uma das M linhas seguintes contém três inteiros u , v e k que descrevem uma trilha entre as casas u e v e a quantidade de k monstros nela.

Saída Para cada caso de teste, o programa deve produzir uma linha na saída contendo S se Jack Lanterna consegue eliminar todos os monstros e retornar à sua casa inicial, ou N caso contrário.

Exemplos de Entrada	Exemplos de Saída
5 6 1 2 1 1 3 2 1 4 3 1 5 4 2 3 2 4 5 1	S
5 7 1 2 1 1 3 2 1 4 3 1 5 4 2 3 2 3 4 1 4 5 1	N
6 6 1 2 1 1 3 2 2 3 3 4 5 1 5 6 2 6 4 3	N

Table 26. Exemplos de entradas e saídas

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4  vector<int> cor;
5  vector<int> pi;
6
7  void leituraGrafo(vector<vector<double>> &G, int m)
8  {
9      int a, b;
10     double c;
11     while (m--)
12     {
13         cin >> a >> b >> c;
14         G[a - 1][b - 1] = c;
15         G[b - 1][a - 1] = c;
16     }
17 }
18
19 int vizinhos(vector<vector<double>> &G)
20 {
21
22     int n = G.size();
23     for (int u = 0; u < n; u++)
24     {
25         int c = 0;
26         for (int v = 0; v < n; v++)
27         {
28             if (G[u][v] > 0)
29             {
30                 c++;
31             }

```



```
32     }
33     if (c % 2 == 1)
34     {
35         return 0;
36     }
37 }
38 return 1;
39 }
40
41 void dfsvisit(int u, vector<vector<double>> &G)
42 {
43     cor[u] = 1;
44     for (auto v : G[u])
45     {
46         if (cor[v] == 0)
47         {
48             pi[v] = u;
49             dfsvisit(v, G);
50         }
51     }
52     cor[u] = 2;
53 }
54 int dfs(vector<vector<double>> &G)
55 {
56     int n = G.size();
57     cor.assign(n, 0);
58     pi.assign(n, -1);
59     int u, c = 0;
60     for (u = 0; u < n; u++)
61     {
62         if (cor[u] == 0)
63         {
64             c++;
65             dfsvisit(u, G);
66         }
67     }
68     if (c == 1)
69     {
70         return 1;
71     }
72     else
73     {
74         return 0;
75     }
76 }
77 int main()
78 {
79     int n, m;
80     cin >> n >> m;
81     vector<vector<double>> Grafo;
82     Grafo.assign(n, vector<double>(n, 0));
83     leituraGrafo(Grafo, m);
84     int ok = 0;
85     if (dfs(Grafo))
86     {
87         ok++;
88     }
89     if (vizinhos(Grafo))
90     {
91         ok++;
92     }
93     if (ok == 2)
94     {
95         cout << "S" << endl;
96     }
97     else
98     {
```

```
99         cout << "N" << endl;  
100     }  
101     return 0;  
102 }
```

Problema C

Halloween no Bairro

Nome do arquivo: C.c, C.cpp, C.java, ou C.py

Timelimit: 1

Por André da Cunha Ribeiro

Neste Halloween do Abóbora Contest 5, várias crianças decidiram sair para pedir doces em seu bairro. Cada criança tem um número de identificação de 1 a N , e todas as casas no bairro distribuem doces em uma ordem aleatória. Ao final da noite, cada criança retorna para contar quantos doces coletou e, com base nesses números, você deve determinar a posição de cada criança no ranking de doces recebidos.

Por exemplo, se $N = 5$ e as quantidades de doces coletados pelas crianças foram: 8, 15, 7, 20, 10, então a criança com número 1 ficou na 4ª posição, a criança com número 2 ficou na 2ª posição, a criança com número 3 ficou na 5ª posição, a criança com número 4 ficou na 1ª posição e a criança com número 5 ficou na 3ª posição.

Sua tarefa é ajudar a classificar as crianças de acordo com a quantidade de doces recebidos, da seguinte forma: a criança que coletou mais doces deve ficar na 1ª posição, a que coletou o segundo maior número de doces fica na 2ª posição, e assim por diante.

Entrada A primeira linha da entrada contém um único inteiro N , representando o número de crianças que participaram. Nas próximas linhas contêm N inteiro, representando a quantidade de doces coletados pela criança de número correspondente.

- Cada criança coleta uma quantidade diferente de doces.
- $1 \leq N \leq 10.000$;

Saída Seu programa deve imprimir um linha contendo os N inteiro separados por um espaço. A i -ésima posição deve conter a posição no ranking da criança com número i . O último valor não tem espaço em branco, somente o terminador de linha.

Exemplos de Entrada	Exemplos de Saída
5 8 15 7 20 10	4 2 5 1 3
3 12 5 8	1 3 2

Table 27. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2
3  void troca(int *a, int *b)
4  {
5      int aux;
6      aux = *a;
7      *a = *b;
8      *b = aux;
9  }
10 int main()
11 {
12     int N;
13     scanf("%d", &N);
14     int doces[N];
15     int ordem[N];
16
17     for (int i = 0; i < N; i++)
18     {
19         scanf("%d", &doces[i]);
20         ordem[i]=i;
21     }
22
23
24     for (int i = 0; i < N; i++)
25     {
26         for (int j = i + 1; j < N; j++)
27         {
28             if (doces[i] < doces[j])
29             {
30                 troca(&doces[j], &doces[i]);
31                 troca(&ordem[i], &ordem[j]);
32             }
33         }
34     }
35
36     for (int i = 0; i < N; i++)
37     {
38         printf("%d",ordem[i]+1);
39         if (i != (N - 1))
40             printf(" ");
41         else printf("\n");
42     }
43
44     return 0;
45 }
```

Problema D

Balança do Festival

Nome do arquivo: D.c, D.cpp, D.java, ou D.py

Timelimit: 1

Por Athos José

Na cidade das Abóboras, Zé do Grafo é famoso por fazer suas peripécias com grafo, mas agora ele quer cultivar as mais saborosas abóboras da região. Com a chegada do *Concurso das Abóboras*, ela precisa escolher algumas de suas abóboras para exibir no festival. No entanto, existe uma regra: a soma dos pesos das abóboras escolhidas deve ser um valor P estabelecido pelo festival.

Zé do Grafo já colheu algumas abóboras e está determinado a participar, mas precisa da sua ajuda para decidir se ele conseguirá escolher algumas abóboras que atinjam exatamente o peso estabelecido, sem passar nem faltar. Será que você consegue ajudá-lo dizendo se ele tem essas abóboras?

Entrada

A primeira linha contém dois inteiros, N , P , representando respectivamente o número de abóboras ($1 \leq N \leq 10^4$) e o peso para competição ($1 \leq P \leq 10^4$). A segunda linha contém N inteiros $a_1, \dots, a_N$ ($1 \leq a_i \leq 100$), onde a_i indica o peso da abóbora.

Saída

Imprima “SIM” se for possível escolher algumas abóboras de modo que a soma dos pesos delas seja exatamente P . Caso contrário, imprima “NAO”.

Exemplos de Entrada	Exemplos de Saída
5 9 3 1 4 2 5	SIM
4 11 8 4 6 2	NAO

Table 28. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #define NMAX 10001
5  #define MMAX 10001
6
7  int abobora[NMAX];
8  int dp[NMAX][MMAX];
9
10 int solver(int n, int sum){
11     for (int i = 0; i <= n; i++)
12         dp[i][0] = 1;
13     for (int i = 1; i <= n; i++) {
14         for (int j = 1; j <= sum; j++) {
15             if (j < abobora[i - 1]) {
16                 dp[i][j] = dp[i - 1][j];
17             } else{
18                 dp[i][j] = dp[i - 1][j] || dp[i - 1][j - abobora[i - 1]];
19             }
20         }
21     }
22     return dp[n][sum];
23 }
24
25 int main() {
26     int p, n;
27     int x;
28     cin >> n >> p;
29     memset(dp, 0, sizeof(dp));
30     for (int i = 0; i < n; i++){
31         scanf("%d", &x);
32         abobora[i] = x;
33     }
34     if (solver(n, p))
35         printf("SIM\n");
36     else
37         printf("NAO\n");
38     return 0;
39 }
```

Problema E

Estratagema Abobrense

Nome do arquivo: E.c, E.cpp, E.java, ou E.py

Timelimit: 1

Por Márcio Belo

No dia dos mortos, as crianças abobrenses costumam pintar os gomos de abóboras maduras, nunca pintando gomos vizinhos/adjacentes e cada gomo com uma cor diferente. Por exemplo, a abóbora da figura abaixo possui 10 gomos, sendo que 2 gomos foram pintados com 2 cores diferentes, com 1 gomo não pintado entre eles (pelo outro sentido, 7 gomos).

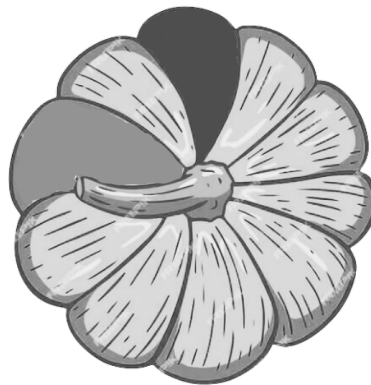


Figure 5. Abóbora de 10 gomos, com 2 gomos pintados.

O professor Márcio Belo, intrigado com a cultura local, decidiu calcular de quantas formas é possível pintar P gomos de uma abóbora com N gomos, de P cores diferentes, e considerando que entre dois gomos pintados devem haver pelo menos R gomos não pintados. Considere também que os gomos são diferentes entre si. Você pode ajudar o professor?

Entrada Cada entrada contém uma única linha com: o número N de gomos da abóbora, com ($2 \leq N \leq 10^6$); o número P de gomos a ser pintados e o número de cores diferentes, com ($1 \leq P \leq N$); e o número R de mínimo de gomos não pintados entre gomos pintados ($1 \leq R < N$).

Saída O sistema deve retornar a quantidade de formas em que, dada uma abóbora de N gomos é possível pintar P gomos com cores diferentes com no mínimo R gomos não pintados entre dois gomos pintados. Quando o problema for impossível, retorne 0. A resposta deve ser dada módulo $10^9 + 7$.

Exemplos de Entrada	Exemplos de Saída
10 1 1	10
10 2 1	70
10 5 1	240
10 6 1	0
10 2 2	50
100 24 2	64268812

Table 29. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      const long long md = 1000000007LL;
9      long long N, P, R;
10     cin >> N >> P >> R;
11
12     long long KC;
13     if (P * R + P > N) {
14         KC = 0;
15     } else {
16         KC = N % md;
17         for (long long i = 1; i < P; i++) {
18             KC = (KC * (N - P * R - i)) % md;
19         }
20     }
21
22     cout << KC << "\n";
23     return 0;
24 }
```


Problema F

A teia da aranha Tecelã

Nome do arquivo: F.c, F.cpp, F.java, ou F.py

Timelimit: 1

Por Douglas Cedrim

A Tecelã é uma aranha que vive antenada nas novas tecnologias que surgem pra ajudar no seu cotidiano. No seu dia a dia ela produz uma certa quantidade X de fibra, que será utilizada como matéria prima nas teias que ela tece. Essa teia é muito importante pois permite, por exemplo, que ela consiga capturar insetos para sua própria alimentação.

Ao visitar o prédio de Informática no IF Goiano, ela observou vários tubos onde poderia tecer sua teia para buscar alimento. Ela observou que todos eles tinham o formato cilíndrico, com as extremidades definidas por círculos. Decidiu então que só iria tecer uma teia se a quantidade X de fibras que ela produziu, que é medida em m^2 , fosse suficiente para cobrir toda a secção transversal da extremidade do tubo, e que precisaria bolar um algoritmo para isso. O problema é que os tubos têm tamanhos bem diferentes entre si.

Sendo assim, ela vai a um determinado tubo e percorre o contorno da secção transversal, produzindo uma teia que mede exatamente o comprimento C da circunferência do cilindro. Considere que a espessura do cilindro é irrelevante para as contas. E também que o quanto ela usou de teia para a medição (C) não irá impactar na decisão.

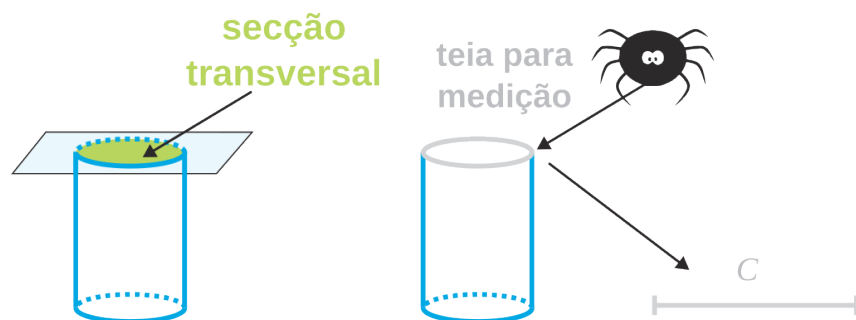


Figure 6. Esquema do algoritmo da aranha Tecelã

E aí, será que você pode ajudar a Tecelã a identificar se ela deve tecer a sua teia em um determinado tubo?

Entrada Cada entrada contém duas medidas, separadas por um espaço: primeiro o montante X de fibra produzido pela Tecelã, em m^2 ; e em seguida o comprimento C da circunferência do tubo, medido com sua teia.

Saída O sistema deve retornar "SIM" caso a quantidade de fibra que ela possui seja suficiente para cobrir a secção da extremidade do tubo. Caso contrário, a resposta deverá ser "NAO". Não esqueça do terminador de quebra de linha.

Exemplos de Entrada	Exemplos de Saída
3.4 6.3	SIM
3.0 6.3	NAO
0.09 1	SIM

Table 30. Exemplos de entradas e saídas

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     float total_teia, comprimento;
9     const float pi = 3.14f;
10
11     while(cin >> total_teia >> comprimento){
12         float area = (comprimento * comprimento) / (4.0f * pi);
13         if(total_teia <= area)
14             cout << "NAO\n";
15         else
16             cout << "SIM\n";
17     }
18
19     return 0;
20 }
```

Problema G

Zé do Grafo no Halloween

Nome do arquivo: G.c, G.cpp, G.java, ou G.py

Timelimit: 1

Por André da Cunha Ribeiro

Zé do Grafo, o famoso bruxo das noites de Halloween e do Abóbora Contest, adora brincar com suas poções mágicas e encantamentos geométricos. Este ano, ele está experimentando sua força com palitinhos mágicos, que têm o poder de se transformar em lados de um triângulo. No entanto, para garantir que o feitiço funcione, ele precisa seguir a regra mágica da **Desigualdade Triangular**, ou seja, a soma dos comprimentos de quaisquer dois palitinhos deve ser maior que o comprimento do terceiro.

Zé do Grafo possui vários palitinhos retos de diversos tamanhos e o objetivo é formar um triângulo com eles pela regra mágica, de modo que cada palitinho represente exatamente um lado do triângulo. Não é permitido que algum lado possua um buraco, nem que um pedaço de algum palitinho sobre para fora do triângulo. A figura 7 ilustra um trio permitido na sub-figura (a) e as sub-figuras (b) e (c) ilustram trios proibidos de acordo com as regras mágica.

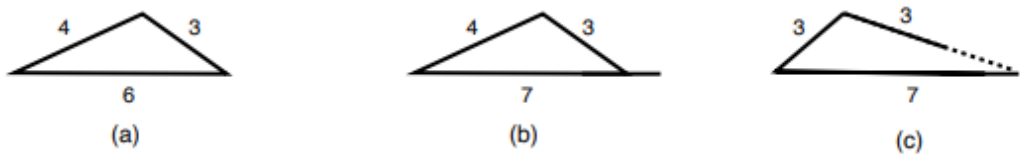


Figure 7. Exemplos de palitinhos

Zé do Grafo quer saber se com três palitinhos distintos, ele consegue um triângulo pela regra mágica.

Entrada A entrada são três números inteiros, que representa o tamanho de cada palitinho.

Saída O programa deve retornar “SIM” caso Zé do Grafos consiga forma o triângulo. Caso contrário, a resposta será “NAO”. Não esqueça do terminador de linha.

Exemplos de Entrada	Exemplos de Saída
3 4 6	SIM
3 4 7	NAO
3 3 7	NAO

Table 31. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2
3  int main()
4  {
5      int a, b, c;
6      scanf("%d %d %d", &a, &b, &c);
7      if ((a < b + c) && (b < a + c) && (c < a + b))
8      {
9          printf("SIM\n");
10     }
11     else
12     {
13         printf("NAO\n");
14     }
15     return 0;
16 }
```

Problema H

Josephus das Abóboras

Nome do arquivo: H.c, H.cpp, H.java, ou H.py

Timelimit: 1

Por Heverton Barros de Macêdo

Finalmente chegou o dia das bruxas e os “corajosos” estudantes de Ciência da Computação estão prontos para adentrar no castelo mal-assombrado. O problema é que a porta do castelo é muito estreita e permite que apenas um estudante adentre por vez. Como todos estão com medo, Josephus decidiu fazer um sorteio para estabelecer a ordem em que eles devem entrar. A seguinte regra foi estabelecida:

1. Os estudantes devem se posicionar em formato de círculo;
2. Cada estudante apresenta um número com os dedos da mão;
3. Faz-se a soma dos números e define o resultado como X ;
4. A contagem dos estudantes é iniciada em zero a partir do primeiro estudante adicionado no círculo, seguindo em sentido horário. Quando a contagem atingir X , o estudante daquela posição será escolhido como o primeiro a ingressar no castelo.
5. O estudante seguinte ao escolhido, servirá como ponto de partida para a nova contagem até o número X , que determina o próximo a ingressar no castelo.
6. O passo anterior deverá ser repetido até que reste apenas um estudante, momento em que a sequência é finalizada, definindo o último aluno a ingressar no castelo.

Como exemplo, considere os estudantes: Fulano, Beltrano, Sicrano e Josephus. A disposição dos estudantes colocando-os em círculo pode ser visualizada na Figura 8.

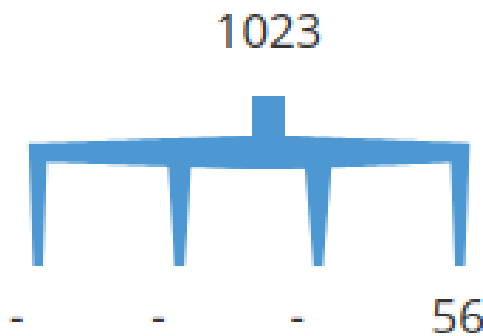


Figure 8. Disposição inicial em formato circular

Considere que a soma dos números dos dedos da mão indicados pelos estudantes seja 7 ($X = 7$). Fazendo a contagem no sentido horário, partindo de Fulano, temos: Beltrano (1), Sicrano (2), Josephus (3), Fulano (4), continuando o círculo, Beltrano (5), Sicrano (6), Josephus (7). A Figura 9 ilustra a contagem, destacando o número 7 em vermelho como sendo o estudante a ser removido do círculo.

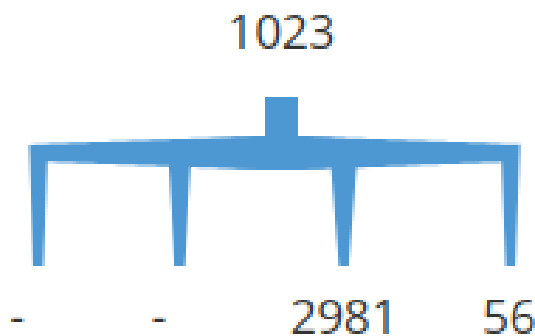


Figure 9. Contagem considerando o número $X = 7$ e o aluno Fulano como referência para o início da contagem

Para essa rodada o Josephus foi selecionado como primeiro a ingressar no castelo. Considerando Josephus fora do círculo, iniciamos novamente a contagem a partir do próximo do círculo (coincidentemente Fulano). Na sequência temos: Beltrano (1), Sicrano (2), Fulano (3), Beltrano (4), Sicrano (5), Fulano (6), Beltrano (7), indicando que Beltrano é o segundo a adentrar no Castelo. Continuando o sorteio com

Beltrano fora do círculo e a contagem iniciando de Sicrano (próximo no sentido horário) teremos: Fulano (1), Sicrano (2), Fulano (3), Sicrano (4), Fulano (5), Sicrano (6), Fulano (7). Por fim, temos a sequência de estudantes a entrar no castelo: Josephus, Beltrano, Fulano e Sicrano.

Sua missão é ajudar Josephus a obter a sequência correta para adentrar ao castelo, independente do número de estudantes que estiver o acompanhando.

Entrada: A entrada contém vários casos de teste. A primeira linha de cada caso contém os nomes dos estudantes separados com um espaço. A segunda linha contém o número inteiro X ($1 \leq X \leq 10.000$) que será usado para a contagem (Obs: para cada nome não deve haver espaço e haverá no máximo 20 caracteres. Exemplo de nome inválido: "João da Silva". Exemplo de nome válido "JoãodaSilva").

Saída: O programa deve produzir como saída a sequência de nomes a adentrar no castelo separados por um espaço (Obs: Ao término do último nome da lista não deve haver espaço, somente o terminador de linha).

Exemplos de Entrada	Exemplos de Saída
Fulano Sicrano 1	Sicrano Fulano
Fulano Sicrano Beltrano 1	Sicrano Fulano Beltrano
Fulano Sicrano Beltrano 2	Beltrano Fulano Sicrano
Fulano Sicrano Beltrano 6	Fulano Sicrano Beltrano
Fulano Beltrano Sicrano Josephus 7	Josephus Beltrano Fulano Sicrano

Table 32. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct No{
5      string nome;
6      No *prox;
7  };
8
9  struct Desc{
10     int qtd;
11     No *ini, *fim;
12 };
13
14 void inserir(Desc *d, const string &nome){
15     No *novo = new No;
16     novo->nome = nome;
17     novo->prox = nullptr;
18
19     if(d->ini == nullptr){
20         d->ini = novo;
21         d->fim = novo;
22         novo->prox = novo;
23     }else{
24         novo->prox = d->ini;
25         d->fim->prox = novo;
26         d->fim = novo;
27     }
28     d->qtd++;
29 }
30
31 void mostrar(Desc *d){
32     if(d->ini != nullptr){
33         No *p;
34         for(p = d->ini; p->prox != d->ini; p = p->prox){
35             cout << p->nome << " ";
36         }
37         cout << p->nome;
38     }
39 }
40
41 int remover_ultimo(Desc *d){
42     if(d->ini == nullptr) return 0;
43
44     if(d->ini == d->fim){
45         delete d->fim;
46         d->ini = nullptr;
47         d->fim = nullptr;
48     }else{
49         No *p;
50         for(p = d->ini; p->prox != d->fim; p = p->prox);
51         p->prox = d->ini;
52         delete d->fim;
53         d->fim = p;
54     }
55     d->qtd--;
56     return 1;
57 }
58
59 void processar_listas(Desc *d, Desc *r, int x){
60     if(d->ini != nullptr){
61         No *p = d->ini, *q = nullptr;
62
63         d->ini = nullptr;
64         d->fim = nullptr;
65
66         while(p->prox != p){
```

```
67         for(int i = 0; i < x % d->qtd; i++, p = p->prox){
68             q = p;
69         }
70
71         q->prox = p->prox;
72
73         if(r->ini == nullptr){
74             r->ini = p;
75             r->fim = p;
76             p->prox = p;
77         }else{
78             r->fim->prox = p;
79             r->fim = p;
80             p->prox = r->ini;
81         }
82
83         r->qtd++;
84         d->qtd--;
85         p = q->prox;
86     }
87
88     r->fim->prox = p;
89     r->fim = p;
90     p->prox = r->ini;
91
92     r->qtd++;
93     d->qtd--;
94 }
95 }
96
97 int main(){
98     ios::sync_with_stdio(false);
99     cin.tie(nullptr);
100
101     Desc d, r;
102     d.ini = nullptr;
103     d.fim = nullptr;
104     d.qtd = 0;
105     r.ini = nullptr;
106     r.fim = nullptr;
107     r.qtd = 0;
108
109     string nome;
110     while(cin >> nome){
111         inserir(&d, nome);
112     }
113
114     int x = atoi(d.fim->nome.c_str());
115     remover_ultimo(&d);
116
117     processar_listas(&d, &r, x);
118     mostrar(&r);
119
120     while(remover_ultimo(&r));
121
122     return 0;
123 }
```




6º Abóbora Contest & 3ª Maratona de Programação IF

30 de Setembro de 2025

Instruções para uso do sistema de submissão Boca

Adaptado de ICPC Latin American Regional Contests, 2024.

1. Esclarecimentos (*clarifications*)

Toda a comunicação com os juizes é feita através das mensagens de **esclarecimentos**. Para solicitar um esclarecimento sobre algum problema você deverá utilizar a interface *web* do Boca:

- (a) Abra seu navegador;
- (b) Efetue login como time;
- (c) Acesse a opção *clarifications*. Escolha o problema sobre o qual você necessita esclarecimento, digite sua pergunta e submeta.

Tanto os pedidos de esclarecimento enviados por vocês quanto as respostas dos juizes estarão disponíveis nessa mesma tela. Haverá um alerta pela interface *web* sempre que você obtiver resposta para algum pedido de esclarecimento. Caso o juiz entenda que o seu esclarecimento se aplica aos demais, será feito um esclarecimento global.

2. Placar (*score board*)

Para visualizar o placar (*score board*), onde você poderá acompanhar o *ranking* entre os times, você deverá utilizar a interface *web* do Boca:

- (a) Abra seu navegador;
- (b) Efetue login como time;
- (c) Acesse a opção *Score*. Você terá acesso ao *ranking* completo.

3. Tarefas (*tasks*)

A aba *Tasks* permite que o time envie códigos para impressão, bem como solicitar ajuda de algum membro do suporte (*staff*).

- (a) Abra seu navegador;
- (b) Efetue login como time e vá até a aba *Tasks*;
- (c) Para imprimir um arquivo selecione ele no disco e clique em enviar (*Send*);
- (d) Para solicitar ajuda, clique no botão *S.O.S.*. Note que a ajuda feita pelo suporte terá relação *apenas* com problemas na máquina ou outro problema físico.

6º Abóbora Contest & 3ª Maratona de Programação IF

30 de Setembro de 2025

Caderno de Problemas

Informações Gerais

Este caderno contém 7 problemas; as páginas estão numeradas de 1 a 8, contando esta página de rosto. Verifique se o caderno está completo.

1. Sobre os nomes dos programas

- (a) Para soluções em C/C++ e Python 3.10, o nome do arquivo-fonte não é significativo, pode ser qualquer nome desde que tenha as extensões .c, .cpp, .java e .py.
- (b) Para linguagem Java, sua solução deve ser chamado de `codigo_do_problema.java`, em que `codigo_do_problema` é a letra maiúscula que identifica o problema. Lembre que em Java o nome da classe principal deve ser igual ao nome do arquivo.

2. Sobre a entrada

- (a) A entrada de seu programa deve ser lida da entrada padrão.
- (b) A entrada é composta de um único caso de teste, descrito em um número de linhas que depende do problema.
- (c) Quando uma linha da entrada contém vários valores, estes são separados por um único espaço em branco; a entrada não contém nenhum outro espaço em branco.
- (d) Cada linha, incluindo a última, contém exatamente um caractere final-de-linha.
- (e) O final da entrada coincide com o final do arquivo.

3. Sobre a saída

- (a) A saída de seu programa deve ser escrita na saída padrão.
- (b) Quando uma linha da saída contém vários valores, estes devem ser separados por um único espaço em branco; a saída não deve conter nenhum outro espaço em branco.
- (c) Cada linha, incluindo a última, deve conter exatamente um caractere final-de-linha.

Problema A

Caminho da Colheita

Nome do arquivo: A.c, A.cpp, A.java, ou A.py

Timelimit: 1

Por Athos José

Seu Zé, fazendeiro importante de Rio Verde tem uma plantação de abóboras organizadas em uma área retangular de tamanho $N \times M$, cada muda de abóbora está em uma área 1×1 e contém uma certa quantidade de abóboras.

Costumeiramente, Seu Zé faz a colheita em um percurso único, começando no canto superior esquerdo do campo (posição $(1, 1)$) e termina sempre no canto inferior direito do campo (N, M) . A cada passo, ele mover-se apenas para a direita ou para baixo.

A plantação de Seu Zé cresceu muito e acredita que o carinho que usa para colher não é suficientemente grande para acomodar todas as abóboras no caminho. Agora ele pediu sua ajuda para que dado o campo informa-se qual é máximo de abóboras possível de colher ao longo do melhor caminho.

	1	2	3	4
1	5	2	3	1
2	1	8	1	1
3	4	2	1	10

Figure 10. Exemplo da entrada com células coloridas

Entrada

A primeira linha contém dois inteiros N e M ($1 \leq N, M \leq 2000$), representando respectivamente o número de linhas e colunas da grade.

Em seguida, seguem N linhas, cada uma contendo M inteiros $C_{i,j}$ ($0 \leq C_{i,j} \leq 10^4$), representando a quantidade de abóboras em cada posição i e j .

Saída

Imprima um único inteiro, sendo o número máximo de abóboras que podem ser colhidas seguindo as regras.

Exemplos de Entrada	Exemplos de Saída
3 4 5 2 3 1 1 8 1 1 4 2 1 10	28

Table 33. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  #define TAM 2001
6  int campo[TAM][TAM];
7  unsigned long dp[TAM][TAM];
8
9  void solver(int n, int m){
10     dp[0][0] = campo[0][0];
11     for(int i = 1; i < n; i++){
12         dp[i][0] = dp[i-1][0]+campo[i][0];
13     }
14     for(int j = 1; j < m; j++){
15         dp[0][j] = dp[0][j-1]+campo[0][j];
16     }
17     for(int i = 1; i < n; i++){
18         for(int j = 1; j < m; j++){
19             dp[i][j] = max(dp[i-1][j]+campo[i][j], dp[i][j-1]+campo[i][j]);
20         }
21     }
22 }
23
24 int main(){
25     int n, m;
26     cin >> n >> m;
27     for (int i = 0; i < n; i++){
28         for(int j = 0; j < m; j++){
29             cin >> campo[i][j];
30         }
31     }
32     solver(n, m);
33     cout << dp[n-1][m-1] << "\n";
34     return 0;
35 }
```

Problema B

O Desafio do Chocolate do Zé do Grafo

Nome do arquivo: B.c, B.cpp, B.java, ou B.py

Timelimit: 1

Por André da Cunha Ribeiro

Todo competidor de maratona de programação sabe que as longas horas de um concurso exigem um combustível especial para o cérebro. E qual é o lanche preferido de 10 entre 10 programadores para manter o foco e a energia? Chocolate, é claro! A combinação de açúcar e cafeína parece ter sido feita sob medida para transformar um bug teimoso em um cobijado "Accepted".

Zé do Grafo, um famoso professor de Teoria dos Grafos e um ávido criador de quebra-cabeças, propôs um desafio para seus alunos. Ele apresentou uma barra de chocolate retangular, dividida em pequenos quadrados por sulcos, formando uma grade de N fileiras e M colunas.

O desafio é o seguinte: para dividir a barra de chocolate em $(N * M)$ quadrados de chocolate individuais, os alunos só podem realizar um tipo de movimento. A cada passo, eles podem pegar um único pedaço de chocolate e quebrá-lo ao longo de um dos sulcos (seja na horizontal ou na vertical), resultando em dois pedaços menores.

Sua tarefa é criar um programa que ajude os alunos do Zé do Grafo a descobrir rapidamente o número total de quebras necessárias para transformar a barra inteira em quadrados individuais.

Entrada A entrada contém na primeira linha o valor $1 \leq X \leq 1000$ que representa o número dos casos de teste. Seguido de X leituras dos valores de N, M ($1 \leq N, M \leq 10^9$) indicando as dimensões dos chocolates.

Saída Para cada caso de teste, o programa deve produzir uma linha na saída, contendo um único inteiro que representa o número total de quebras necessárias para dividir o chocolate daquele caso de teste.

Exemplos de Entrada	Exemplos de Saída
4	3
2 2	23
4 6	55
7 8	836
27 31	

Table 34. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main()
4  {
5      int x;
6      cin >> x;
7      while (x--)
8      {
9          long n, m;
10         cin >> n >> m;
11         long resultado = (n * m) - 1;
12         cout << resultado << endl;
13     }
14     return 0;
15 }
```

Problema C

Otimização de Rotas de Coleta de Solo - Simple Smart Agro

Nome do arquivo: C.c, C.cpp, C.java, ou C.py

Timelimit: 2

Por Alexandre da Silva

A Simple Smart Agro está desenvolvendo um sistema automatizado para coleta de amostras de solo em uma fazenda. O sistema precisa otimizar as rotas de coleta para maximizar a eficiência do processo. Existem N pontos específicos onde amostras de solo devem ser coletadas. O veículo de coleta parte sempre do depósito localizado na origem $(0, 0)$ e deve retornar ao mesmo ponto após coletar todas as amostras. Devido às características do terreno e dos equipamentos, o veículo só pode se mover nas direções cardeais (norte, sul, leste, oeste) e sempre em linha reta entre dois pontos consecutivos da rota. A distância entre dois pontos (x_1, y_1) e (x_2, y_2) é calculada usando a distância Manhattan: $|x_1 - x_2| + |y_1 - y_2|$.

Entrada

- A primeira linha contém um inteiro N ($1 \leq N \leq 10$), representando o número de pontos de coleta;
- As próximas N linhas contêm dois inteiros x_i e y_i ($-1000 \leq x_i, y_i \leq 1000$), representando as coordenadas do i -ésimo ponto de coleta;
- Todos os pontos de coleta são distintos entre si;
- Nenhum ponto de coleta coincide com a origem $(0, 0)$.

Saída

- Uma única linha contendo um inteiro: a menor distância total (distância Manhattan) necessária para visitar todos os pontos de coleta e retornar à origem.

Exemplos

Exemplo de Entrada	Exemplo de Saída
3 2 3 7 1 5 6	26
4 3 4 8 2 12 8 6 10	42
1 2 2	8

Explicação do Primeiro Exemplo

Partindo da origem $(0, 0)$, temos os pontos de coleta em $(2, 3)$, $(7, 1)$ e $(5, 6)$. Uma rota ótima seria: $(0, 0) \rightarrow (2, 3) \rightarrow (5, 6) \rightarrow (7, 1) \rightarrow (0, 0)$

Calculando as distâncias:

- $(0, 0)$ para $(2, 3)$: $|2 - 0| + |3 - 0| = 2 + 3 = 5$
- $(2, 3)$ para $(5, 6)$: $|5 - 2| + |6 - 3| = 3 + 3 = 6$
- $(5, 6)$ para $(7, 1)$: $|7 - 5| + |1 - 6| = 2 + 5 = 7$
- $(7, 1)$ para $(0, 0)$: $|0 - 7| + |0 - 1| = 7 + 1 = 8$

Distância total: $5 + 6 + 7 + 8 = 26$

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n;
9      cin >> n;
10
11     vector<pair<int,int>> p;
12     p.push_back({0, 0});
13     for(int i = 0; i < n; i++){
14         int x, y;
15         cin >> x >> y;
16         p.push_back({x, y});
17     }
18
19     int m = p.size();
20     vector<vector<long long>> d(m, vector<long long>(m));
21     for(int i = 0; i < m; i++){
22         for(int j = 0; j < m; j++){
23             d[i][j] = llabs(p[i].first - p[j].first) + llabs(p[i].second - p[j].
24                 second);
25         }
26     }
27
28     int maxmask = 1 << m;
29     const long long INF = LLONG_MAX / 4;
30     vector<vector<long long>> dp(maxmask, vector<long long>(m, INF));
31
32     dp[1][0] = 0;
33
34     for(int mask = 0; mask < maxmask; mask++){
35         for(int u = 0; u < m; u++){
36             if(dp[mask][u] == INF) continue;
37             for(int v = 0; v < m; v++){
38                 if(mask & (1 << v)) continue;
39                 int new_mask = mask | (1 << v);
40                 dp[new_mask][v] = min(dp[new_mask][v], dp[mask][u] + d[u][v]);
41             }
42         }
43     }
44
45     int full = maxmask - 1;
46     long long ans = INF;
47     for(int i = 1; i < m; i++){
48         if(dp[full][i] != INF){
49             ans = min(ans, dp[full][i] + d[i][0]);
50         }
51     }
52
53     cout << ans << "\n";
54     return 0;
55 }
```


Problema D

Balança do Festival

Nome do arquivo: D.c, D.cpp, D.java, ou D.py

Timelimit: 1

Por Athos José

Na cidade das Abóboras, Zé do Grafo é famoso por fazer suas peripécias com grafo, mas agora ele quer cultivar as mais saborosas abóboras da região. Com a chegada do *Concurso das Abóboras*, ela precisa escolher algumas de suas abóboras para exibir no festival. No entanto, existe uma regra: a soma dos pesos das abóboras escolhidas deve ser um valor P estabelecido pelo festival.

Zé do Grafo já colheu algumas abóboras e está determinado a participar, mas precisa da sua ajuda para decidir se ele conseguirá escolher algumas abóboras que atinjam exatamente o peso estabelecido, sem passar nem faltar. Será que você consegue ajudá-lo dizendo se ele tem essas abóboras?

Entrada

A primeira linha contém dois inteiros, N , P , representando respectivamente o número de abóboras ($1 \leq N \leq 10^4$) e o peso para competição ($1 \leq P \leq 10^4$). A segunda linha contém N inteiros $a_1, \dots, a_N$ ($1 \leq a_i \leq 100$), onde a_i indica o peso da abóbora.

Saída

Imprima “SIM” se for possível escolher algumas abóboras de modo que a soma dos pesos delas seja exatamente P . Caso contrário, imprima “NAO”.

Exemplos de Entrada	Exemplos de Saída
5 9 3 1 4 2 5	SIM
4 11 8 4 6 2	NAO

Table 35. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #define NMAX 10001
5  #define MMAX 10001
6
7  int abobora[NMAX];
8  int dp[NMAX][MMAX];
9
10 int solver(int n, int sum){
11     for (int i = 0; i <= n; i++)
12         dp[i][0] = 1;
13     for (int i = 1; i <= n; i++) {
14         for (int j = 1; j <= sum; j++) {
15             if (j < abobora[i - 1]) {
16                 dp[i][j] = dp[i - 1][j];
17             }else{
18                 dp[i][j] = dp[i - 1][j] || dp[i - 1][j - abobora[i - 1]];
19             }
20         }
21     }
22     return dp[n][sum];
23 }
24
25 int main() {
26     int p, n;
27     int x;
28     cin >> n >> p;
29     memset(dp, 0, sizeof(dp));
30     for (int i = 0; i < n; i++){
31         scanf("%d", &x);
32         abobora[i] = x;
33     }
34     if (solver(n, p))
35         printf("SIM\n");
36     else
37         printf("NAO\n");
38     return 0;
39 }
```

Problema E

Estória da Estônia

Nome do arquivo: E.c, E.cpp, E.java, ou E.py

Timelimit: 1

Por Márcio Belo

O professor Márcio Belo sempre gostou de charadas matemáticas. Um dia ele se deparou com uma estória, pra lá da Estônia. Dado um número primo ímpar p , sempre é possível um triângulo de lados inteiros x , y e z , em ordem crescente de tamanho com as seguintes propriedades:

- A diferença da soma dos lados menores do triângulo pelo lado maior é igual ao primo p ;
- A diferença da soma das áreas dos quadrados de lados x e y e a área do quadrado de lado z é igual ao primo p .

Em matematiqûês:

Dado um primo $p \geq 3$, sempre existem triplas (x, y, z) , com $x \leq y \leq z$ que satisfazem:

$$\begin{aligned}x + y - z &= p \\ x^2 + y^2 - z^2 &= p\end{aligned}$$

Quem são essas triplas (x, y, z) ? Você pode ajudar o professor?

Entrada Cada entrada contém uma única linha com o número primo p , com $(3 \leq p < 2^{16})$.

Saída O sistema deve retornar todas as triplas (x, y, z) em ordem crescente de x . Uma tripla por linha,

Exemplos de Entrada	Exemplos de Saída
3	(4, 6, 7)
5	(6, 15, 16) (7, 10, 12)
13	(14, 91, 92) (15, 52, 54) (16, 39, 42) (19, 26, 32)
65267	(65268, 2129923278, 2129923279) (97900, 130534, 163167)

Table 36. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      long long p;
9      cin >> p;
10
11     long long n = p * (p - 1) / 2;
12     long long lim = sqrt((long double)n);
13
14     for(long long i = 1; i <= lim; i++){
15         if(n % i == 0) {
16             long long a = i;
17             long long b = n / i;
18             cout << "(" << a + p << "," << b + p << "," << a + b + p << ")\n";
19         }
20     }
21
22     return 0;
23 }
```

Problema F

Monitorando Pragas

Nome do arquivo: F.c, F.cpp, F.java, ou F.py

Timelimit: 1

Por Emanuel Silva Araujo

O agricultor Seu Jayme sofre constantemente com infestações de insetos em sua lavoura de soja. Para reduzir perdas e otimizar o manejo, ele procurou a EMBRAPA para desenvolver uma ferramenta de monitoramento diário de pragas. Sua tarefa como programador é implementar um programa que leia os valores de população diária e identifique situações de risco, emitindo alertas de acordo com os seguintes critérios:

- Alerta por pico diário [Pico]
 - Se em qualquer dia a população registrada ultrapassar 200 insetos.
- Alerta por variação em 7 dias [7 Dias]
 - Se a diferença percentual entre o valor mínimo e o máximo em um intervalo de 7 dias consecutivos for maior ou igual a 20%.
- Alerta por crescimento contínuo [Crescimento]
 - Se ocorrerem 4 dias seguidos de crescimento, e em cada dia a população for pelo menos 5% maior que a do dia anterior.

Entrada

Um número inteiro ($0 \leq N \leq 360$), representando a quantidade de dias monitorados (entre os dias 0 e 360). Uma sequência de N números inteiros ($0 < X \leq 5000$), cada um representando a estimativa da população de insetos no respectivo dia.

Saída

Para cada alerta identificado, o programa deve imprimir:

- O tipo do alerta;
- O(s) dia(s) correspondente(s);
- O valor percentual (quando aplicável), com duas casas decimais.

Caso não tenha nenhum alerta, deve imprimir uma mensagem de "Nenhum Alerta". Para dias que tiveram mais de um alerta a ordem de impressão é: [Pico] > [7Dias] > [Crescimento].

Exemplos de Entrada	Exemplos de Saída
10 50 60 70 80 90 200 210 220 180 170	[Crescimento] 1-4: maior que >= 5% [Crescimento] 1-5: maior que >= 5% [Crescimento] 1-6: maior que >= 5% [Pico] 7: 210 insetos [7 Dias] 1-7: 320.00% [Crescimento] 1-7: maior que >= 5% [Pico] 8: 220 insetos [7 Dias] 2-8: 266.67% [7 Dias] 3-9: 214.29% [7 Dias] 4-10: 175.00%
9 100 105 112 118 90 96 100 120 80	[Crescimento] 1-4: maior que >= 5% [7 Dias] 2-8: 33.33%
8 118 120 118 140 140 140 130 100	Nenhum Alerta

Table 37. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int N;
9      cin >> N;
10
11     vector<int> pop(N);
12     for(int i = 0; i < N; i++){
13         cin >> pop[i];
14     }
15
16     int has_alerta = 0;
17     int crescimento = 0;
18
19     for(int i = 0; i < N; i++){
20         if(pop[i] > 200){
21             has_alerta = 1;
22             cout << "[Pico] " << i + 1 << ": " << pop[i] << " insetos\n";
23         }
24
25         if(i >= 6){
26             int minId = i - 6;
27             int maxId = i - 6;
28             for(int j = i - 6; j <= i; j++){
29                 if(pop[j] > pop[maxId]){
30                     maxId = j;
31                 }else if(pop[j] < pop[minId]){
32                     minId = j;
33                 }
34             }
35
36             double diff = (double)(pop[maxId] - pop[minId]) / pop[minId];
37             if(diff >= 0.2 && minId < maxId){
38                 has_alerta = 1;
39                 cout << "[7 Dias] " << i - 5 << "-" << i + 1 << ": ";
40                 cout << fixed << setprecision(2) << diff * 100 << "%\n";
41             }
42         }
43
44         if(i > 0){
45             double diff = (double)(pop[i] - pop[i - 1]) / pop[i - 1];
46             if(diff >= 0.05 && pop[i] > pop[i - 1]){
47                 crescimento++;
48             }else{
49                 crescimento = 0;
50             }
51         }
52
53         if(crescimento >= 3){
54             has_alerta = 1;
55             cout << "[Crescimento] " << i - crescimento + 1 << "-" << i + 1 << ": ";
56             cout << "maior que >= 5%\n";
57         }
58     }
59
60     if(has_alerta == 0){
61         cout << "Nenhum Alerta\n";
62     }
63
64     return 0;
65 }
```

Problema G

Comportamento Dinâmico

Nome do arquivo: G.c, G.cpp, G.java, ou G.py

Timelimit: 1

Por Heverton Barros de Macêdo

Durante uma investigação sobre o comportamento de sistemas dinâmicos, Wolfram definiu o seguinte cenário para efetuar experimentos: dada uma cadeia de N células, linearmente distribuídas, cada célula armazena um único estado (bit), e além disso as células das extremidades são vizinhas entre si. Para cada instante de tempo t a evolução dessa cadeia ocorre simultaneamente em todas as células, considerando uma regra de evolução previamente definida. Essa regra é aplicada a cada vizinhança de $K = 3$ células gerando um novo estado, conforme indicado na Tabela 38.

Table 38. Regra aplicada para evolução do experimento de Wolfram.

Regra para vizinhança	000	001	010	011	100	101	110	111
Novo estado para célula central	1	0	0	1	0	0	0	1

Como exemplo, considere um experimento (veja a Tabela 39) com a cadeia de bits formada por 11 células ($N = 11$), em que no instante $t = 0$ apenas um bit, exatamente no centro da cadeia, possui o estado 1 e as demais células possuem o estado 0, evoluindo por 4 passos de tempo ($t = 4$).

A Tabela 39 ilustra o experimento com a configuração inicial ($t = 0$) formada por: 00000100000. No próximo passo de tempo ($t = 1$), todas as células evoluem (podem alterar o estado) considerando a vizinhança indicada na regra (Tabela 38). Em destaque na cor vermelha (Tabelas 38 e 39) está a atualização da célula central, formada pela vizinhança 010 que, segundo a regra, deve evoluir no próximo passo de tempo ($t = 1$) para o estado 0 (destaque na cor verde). É possível notar no exemplo, a evolução da cadeia inicial por 4 passos de tempo. Vale destacar que as células na extremidade (esquerda e direita) estão conectadas (formato de anel - toroidal)

Table 39. Exemplo de um experimento por 4 passos de tempo.

Tempo	Cadeia linear de N células										
$t = 0$	0	0	0	0	0	1	0	0	0	0	0
$t = 1$	1	1	1	1	0	0	0	1	1	1	1
$t = 2$	1	1	1	0	1	0	0	1	1	1	1
$t = 3$	1	1	0	0	0	0	0	1	1	1	1
$t = 4$	1	0	0	1	1	1	0	1	1	1	1

Sua missão é ajudar Wolfram a obter a configuração dos estados após t passos de tempo, considerando uma sequência inicial qualquer no tempo $t = 0$ e a aplicação da regra indicada no enunciado.

Entrada

A entrada contém vários casos de teste. Cada linha contém um inteiro t ($0 < t \leq 1000$), que representa a quantidade de passos de evolução, seguido pela cadeia de bits no tempo $t = 0$, que pode variar conforme a quantidade de células da amostra N ($3 \leq N < 100$).

Saída

O programa deve produzir como saída a configuração da cadeia no espaço linear de células após a evolução de t passos de tempo.

Exemplos de Entrada	Exemplos de Saída
1 00000100000	11110001111
2 00000100000	11100101111
3 00000100000	11000001111
578 00000100000	00000111101
978 011001111000110111010011	100111000001110000011101

Table 40. Exemplos de entradas e saídas

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void evoluir(string &ret, int qtd){
5      int tam_ret = ret.size();
6      string aux(tam_ret, '0');
7
8      char regra[2][2][2] = {
9          { { '1', '0' }, { '0', '1' } },
10         { { '0', '0' }, { '0', '1' } }
11     };
12
13     for(int total = 0; total < qtd; total++){
14         aux[0] = regra[ret[tam_ret - 1] - '0'][ret[0] - '0'][ret[1] - '0'];
15
16         for(int i = 1; i < tam_ret - 1; i++){
17             aux[i] = regra[ret[i - 1] - '0'][ret[i] - '0'][ret[i + 1] - '0'];
18         }
19
20         aux[tam_ret - 1] = regra[ret[tam_ret - 2] - '0'][ret[tam_ret - 1] - '0'][ret
21             [0] - '0'];
22
23         ret = aux;
24     }
25
26     int main(){
27         ios::sync_with_stdio(false);
28         cin.tie(nullptr);
29
30         int qtd_evolucao;
31         string reticulado;
32
33         while(cin >> qtd_evolucao){
34             cin >> reticulado;
35             evoluir(reticulado, qtd_evolucao);
36             cout << reticulado << "\n";
37         }
38
39         return 0;
40     }
```


References

- Beecrowd. Sort! sort!! and sort!!! - problem 1252, 2025. URL <https://www.beecrowd.com.br/judge/en/problems/view/1252>.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3rd edition, 2009. ISBN 978-0262033848.